

LLM 推理全景透视：综述与 Roofline 模型洞察

Zhihang Yuan^{1,*} Yuzhang Shang^{2,*} Yang Zhou^{3,*} Zhen Dong⁸
Zhe Zhou⁴ Chenhao Xue⁴ Bingzhe Wu⁵ Zhikai Li⁶ Qingyi Gu⁶
Yong Jae Lee⁷ Yan Yan² Beidi Chen³ Guangyu Sun⁴ Kurt Keutzer⁸

¹Infinigence-AI, ²Illinois Institute of Technology, ³Carnegie Mellon University, ⁴Peking University, ⁵Tencent AI Lab,
⁶Institute of Automation, CAS, ⁷University of Wisconsin, Madison, ⁸University of California, Berkeley.

Abstract

高效大语言模型 (LLM) 推理领域正在快速演进, 机遇与挑战并存。尽管该方向发展活跃, 但长期缺少一个足够简明且可操作的统一分析框架, 来帮助研究者系统理解各类 LLM 推理方法。我们的综述不同于传统文献回顾: 不仅总结研究现状, 还提出了一个基于 *Roofline* 模型的系统化分析框架, 用于刻画 LLM 推理中的关键瓶颈。该框架可解释部署中的核心现实问题, 例如: 为何 LLM 往往是 *memory-bound*、其计算与内存需求有多大、如何选择合适硬件。我们系统梳理了高效 LLM 推理的最新进展, 覆盖模型压缩 (如量化)、算法优化 (如 *speculative decoding*)、系统与硬件层增强 (如算子融合) 等关键方向。通过 *Roofline* 视角统一分析这些方法对内存访问与计算的影响, 本综述既刻画研究全景, 也为实际部署提供可落地的参考。因此, 本工作可作为新入门研究者的实践入口, 也可帮助资深研究者深化对高效 LLM 部署问题的理解。分析工具 *LLM-Viewer* 已开源。

1. 引言

近年来, 大语言模型 (LLM) 已成为 AI 发展的核心力量, 持续重塑机器学习与自然语言处理 (NLP) 格局 [Zhao et al., 2023]。这一趋势可追溯到 ChatGPT [Brown et al., 2020, Ouyang et al., 2022] 等里程碑模型的成功, 它们凭借强大的理解与生成能力产出高度拟人化文本。随后, OPT [Zhang et al., 2022]、BLOOM [Scao et al., 2022]、Llama [Touvron et al., 2023a,b] 等模型不断涌现, 进一步强化了“更大模型通常具备更强能力”的共识。因此, 数十亿乃至数百亿参数模型已日益常见。

但模型规模的急剧增长也带来了严峻推理挑战。这些挑战不仅影响算力受限设备, 也同样影响最先进硬件平台。由于模型复杂度高、规模大、能耗与计算成本高, LLM 在真实场景部署难度很大。同时, 高资源消

*Equal contribution. Zhihang Yuan (hahnyuan@gmail.com) is the project leader.

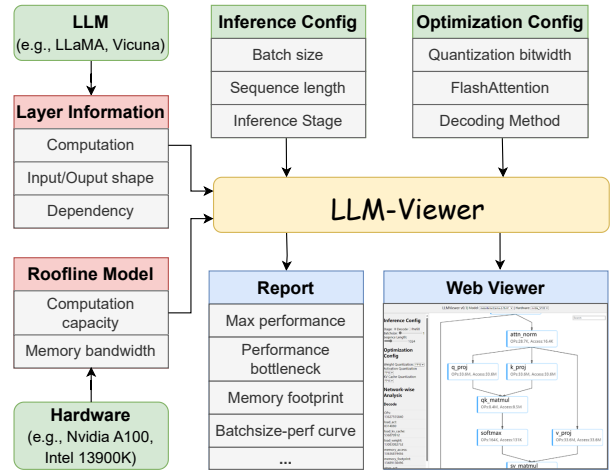


Figure 1. 我们设计的 LLM-Viewer workflow。输入包括目标 LLM 部署配置与硬件设备信息。接收输入后, LLM-Viewer 会精确分析给定 LLM 在指定硬件上的部署瓶颈, 并支持有针对性的高效推理优化。

耗还引发了能效、可扩展性与可及性的担忧, 尤其对中小组织与计算资源有限社区更为不利。因此, 亟需创新方案使 LLM 推理更加普惠与可持续。

为解决部署难题, 大量方法被提出。过去两年, 高效 LLM 推理研究呈指数增长, 机遇与挑战并存。虽然研究繁荣, 但信息量激增也可能掩盖关键趋势、拖慢进一步进展。现有文献的关键缺口在于: 缺乏统一、系统且面向实践的分析框架, 以支撑跨方法对比与综合方案设计。

针对这一缺口, 本文给出了高效 LLM 推理的全景综述, 并强调实践驱动特征。与传统综述不同, 我们不仅归纳已有工作, 还提出专门构建的 Roofline 分析模型, 用于定位 LLM 部署瓶颈 (图 1)。据我们所知, 这是首批面向“LLM 在硬件上的推理复杂性分析”提供工具化支撑的工作之一。我们系统整理了最新进展, 并围绕部署挑战 (特别是推理效率) 展开深入讨论, 覆盖模型压缩、解码算法改进、系统优化与硬件优化 (图 2)。

同时, 近期已有相关综述, 例如 [Zhu et al., 2023] 关注 LLM 压缩, [Miao et al., 2023a]、[Ding et al., 2023]、

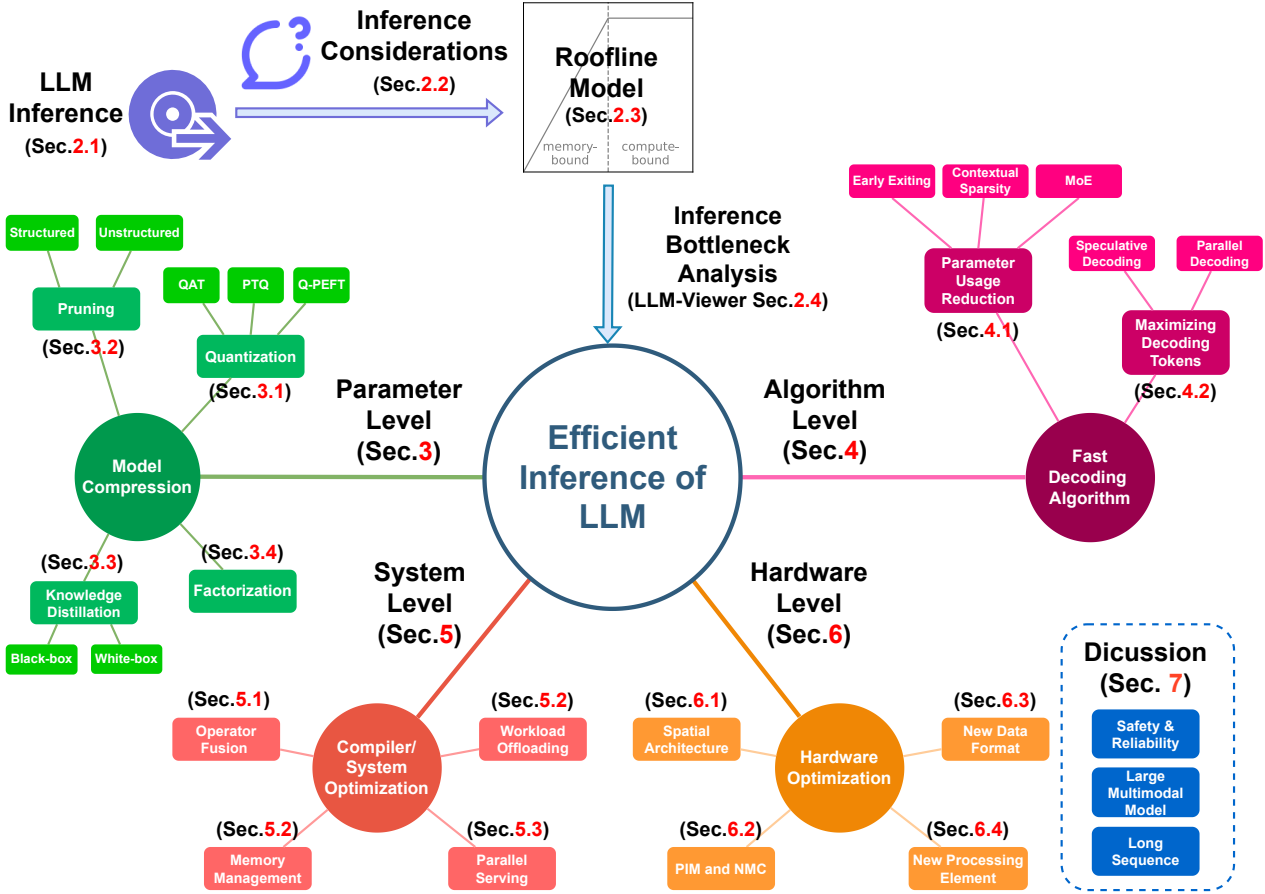


Figure 2. 高效 LLM 推理综述思维导图。本综述不同于传统综述，重点关注 LLM 推理的工程实践问题。我们首先识别并分析 LLM 推理挑战，然后引入定制化 Roofline 模型定位瓶颈 (Sec.2)。随后将推理效率优化分为四个方向：参数压缩 (Sec.3)、快速解码算法 (Sec.4)、系统级优化 (Sec.5) 与硬件级优化 (Sec.6)。

[Wang et al., 2024a] 关注整体服务系统。而我们的差异化在于引入 Roofline 统一分析视角，从而更清晰连接算法机制与部署瓶颈。

本文结构如下：首先介绍 LLM 基础并给出工具 LLM-Viewer (Sec.2)，该工具可基于 Roofline 模型分析任意 LLM 架构在不同硬件平台上的部署瓶颈 (图 1)。随后，本文将高效推理方法归纳为四类：模型压缩 (Sec.3)、快速解码算法 (Sec.4)、编译器/系统级优化 (Sec.5) 以及硬件级优化 (Sec.6)。

2. 深入理解 LLM 推理与部署

2.1. LLM 推理过程

当前大多数大语言模型 (LLM) 采用 Transformer 解码器架构。这里给出其基础结构概览，更深入内容可参考综述 Zhao et al. [2023]。该结构通常由 embedding 层、多层串联 Transformer 层和预测头组成，图 3 给出了示意。

embedding 层将输入 token 映射为隐藏状态；隐藏状态进入 Transformer 层。每个 Transformer 层通常由两部分组成：首先是 masked multi-head attention

(MHA)，随后是 multi-layer perceptron (MLP)。最后一层 Transformer 输出送入 prediction head，以预测下一个 token。

推理与训练相对：训练阶段模型通过数据更新参数；推理阶段参数固定，用户输入 **prompt**，模型生成 **response**。LLM 推理通常分为两阶段：**Prefill** 与 **Decode**。

Prefill 是初始化阶段：模型接收 prompt 序列，并在每个 Transformer 层构建 key-value cache (KV cache)，为后续生成打基础。

在 Prefill 中，MHA 计算并缓存 key/value。设某层输入为 $\mathbf{X}_{\text{pre}} \in \mathbb{R}^{n \times d}$ ，其中 d 为隐藏维度， n 为 prompt 长度；权重为 $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$ 。则：

$$\text{Query: } \mathbf{Q}_{\text{pre}} = \mathbf{X}_{\text{pre}} \cdot \mathbf{W}_q$$

$$\text{Key: } \mathbf{K}_{\text{pre}} = \mathbf{X}_{\text{pre}} \cdot \mathbf{W}_k$$

$$\text{Value: } \mathbf{V}_{\text{pre}} = \mathbf{X}_{\text{pre}} \cdot \mathbf{W}_v$$

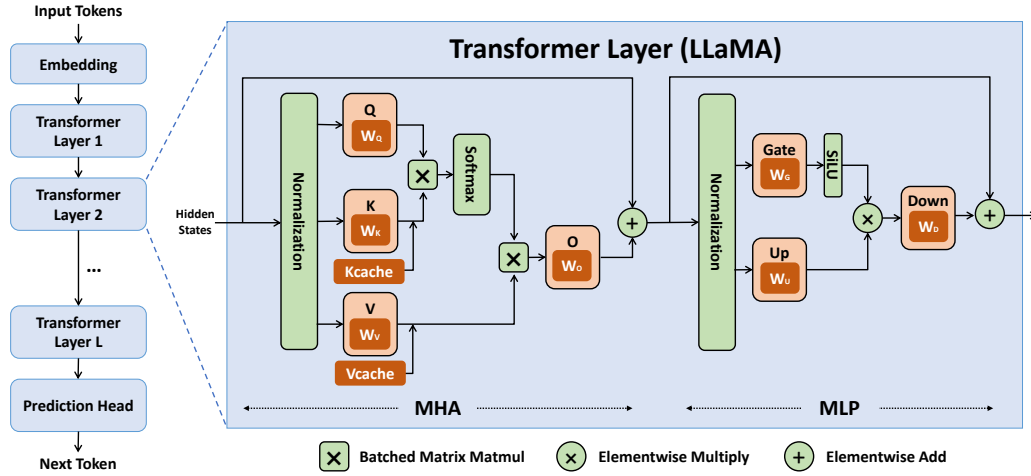


Figure 3. LLM 架构示意图。

$\mathbf{K}_{\text{pre}}$ 与 $\mathbf{V}_{\text{pre}}$ 被写入 KV cache。其余计算可写为¹：

$$\mathbf{O}_{\text{pre}} = \text{softmax} \left(\frac{\mathbf{Q}_{\text{pre}} \cdot \mathbf{K}_{\text{pre}}^T}{\sqrt{d}} \right) \cdot \mathbf{V}_{\text{pre}} \cdot \mathbf{W}_o + \mathbf{X}_{\text{pre}},$$

其中 $\mathbf{O}_{\text{pre}} \in \mathbb{R}^{n \times d}$ 进入 MLP，MLP 输出作为下一层输入。

Decode 是核心生成阶段。模型读取并更新已缓存的 KV 信息，逐 token 生成输出；每个新 token 都依赖之前已生成 token。

在 Decode 中，MHA 读取 $\mathbf{K}_{\text{cache}}$, $\mathbf{V}_{\text{cache}}$ ，输入为 $\mathbf{X}_{\text{dec}} \in \mathbb{R}^{1 \times d}$ ，并拼接新 key/value：

$$\begin{aligned} \text{Query: } \mathbf{Q}_{\text{dec}} &= \mathbf{X}_{\text{dec}} \cdot \mathbf{W}_q \\ \text{Key: } \mathbf{K}_{\text{cat}} &= [\mathbf{K}_{\text{cache}}, \mathbf{X}_{\text{dec}} \cdot \mathbf{W}_k] \\ \text{Value: } \mathbf{V}_{\text{cat}} &= [\mathbf{V}_{\text{cache}}, \mathbf{X}_{\text{dec}} \cdot \mathbf{W}_v] \end{aligned}$$

并执行：

$$\mathbf{O}_{\text{dec}} = \text{softmax} \left(\frac{\mathbf{Q}_{\text{dec}} \cdot \mathbf{K}_{\text{cat}}^T}{\sqrt{d}} \right) \cdot \mathbf{V}_{\text{cat}} \cdot \mathbf{W}_o + \mathbf{X}_{\text{dec}}$$

其中 $\mathbf{O}_{\text{dec}} \in \mathbb{R}^{1 \times d}$ 进入 MLP。最后一层输出送入预测头得到下一个 token。

2.2. Roofline 模型

评估 LLM 在特定硬件上的部署效率，需要同时考虑硬件特性与模型特性。为此我们采用 Roofline 模型。Roofline 是评估模型在目标硬件上理论性能上限的有效工具，可区分 memory bottleneck 与 compute bottleneck。

¹为简洁起见，省略 layer norm、position mask 与 positional embedding。

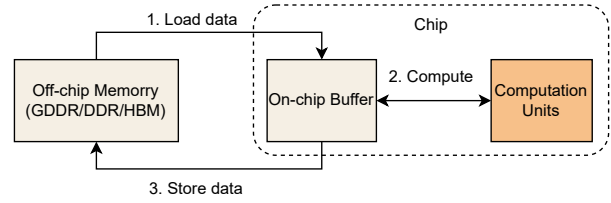


Figure 4. 硬件上执行算子的过程。

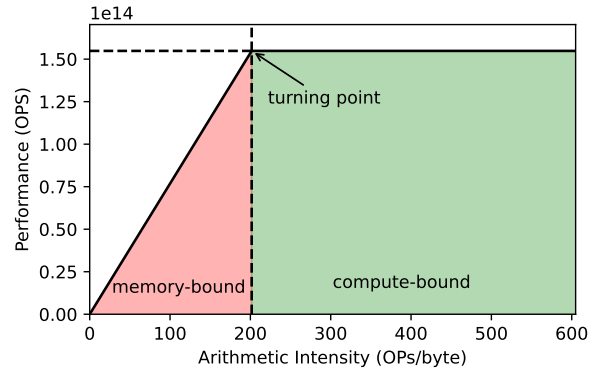


Figure 5. Nvidia A6000 的 Roofline 示意 (FP16 计算)。

如图 4，神经网络层执行包含：数据从 DDR/HBM 读入片上缓冲区，片上计算单元完成计算，再写回内存。因此性能分析必须同时考虑内存访问与计算能力。若某层计算量大、访存少，则是 compute-bound；反之访存重、计算轻，则是 memory-bound。Roofline 能清晰区分二者并给出各场景性能上界。

使用 Roofline 主要两步：

1. 绘制 Roofline：确定目标硬件峰值算力 (OPS)

Table 1. 基于 Nvidia A6000 Roofline 对 Llama-2-7b 各层分析（序列长 2048，batch=1）。

层名	OPs	内存访问	算术强度	最大性能	受限类型
Prefill					
q_proj	69G	67M	1024	155T	compute
k_proj	69G	67M	1024	155T	compute
v_proj	69G	67M	1024	155T	compute
o_proj	69G	67M	1024	155T	compute
gate_proj	185G	152M	1215	155T	compute
up_proj	185G	152M	1215	155T	compute
down_proj	185G	152M	1215	155T	compute
qk_matmul	34G	302M	114	87T	memory
sv_matmul	34G	302M	114	87T	memory
softmax	671M	537M	1.25	960G	memory
norm	59M	34M	1.75	1T	memory
add	8M	34M	0.25	192G	memory
Decode					
q_proj	34M	34M	1	768G	memory
k_proj	34M	34M	1	768G	memory
v_proj	34M	34M	1	768G	memory
o_proj	34M	34M	1	768G	memory
gate_proj	90M	90M	1	768G	memory
up_proj	90M	90M	1	768G	memory
down_proj	90M	90M	1	768G	memory
qk_matmul	17M	17M	0.99	762G	memory
sv_matmul	17M	17M	0.99	762G	memory
softmax	328K	262K	1.25	960G	memory
norm	29K	16K	1.75	1T	memory
add	4K	16K	0.25	192G	memory

与峰值带宽 (bytes/s)²。在坐标中以性能 (OPS) 为 y 轴、算术强度 (OPs/byte) 为 x 轴：绘制峰值算力水平线 (compute roof) 与斜率为峰值带宽的对角线 (memory roofline)。图 5 给出 A6000 示例。

2. **逐层分析**：统计各层 OPs 与内存访问字节数，计算算术强度 (OPs/byte)。再依据第一步图像读取理论性能上界，判断当前点是 memory-bound 还是 compute-bound，以指导后续优化策略。

资源未被充分利用通常有两类情况：当算术强度低于拐点（红区）时，单位访存对应计算量小，即便带宽打满也无法吃满算力，此时为 memory-bound，可优先采用量化、算子融合、增大 batch 等方法缓解访存压力。当算术强度高于拐点（绿区）时，少量访存即可触发大量计算，此时为 compute-bound，可考虑低比特计算等方式提升计算效率。

以表 1 为例，Llama-2-7b 在 A6000 上 prefill 多为 compute-bound、性能较高；decode 全部 memory-bound，性能显著低于 GPU 峰值算力。在交互式场景中 prefill 仅执行一次，而 decode 会反复执行生成连续

²OPS 表示每秒操作数；一次 MAC 记为两次操作。

输出，因此 decode 的 memory-bound 优化是大模型推理加速的核心。

2.3. LLM-Viewer

LLM 由多层 Transformer 组成，每层操作类型复杂，且不同模型操作组合并不一致。同时还需追踪 memory footprint 以估计峰值显存与总推理时延。因此，LLM 分析必须是“网络级”分析而非单算子分析。

我们提出 LLM-Viewer³ 进行网络级分析。它支持在多种硬件平台上分析 LLM 性能与效率，为推理优化提供可解释依据。

LLM-Viewer 工作流（图 1）包括：(1) 输入 LLM，提取各层计算量、输入输出张量形状和数据依赖；(2) 输入硬件参数，构建对应 Roofline 模型（算力与带宽）；(3) 设置推理配置，如 batch size、prompt 长度、生成长度；(4) 设置优化配置，如量化比特、是否使用 FlashAttention、解码策略和系统优化；(5) Analyzer 基于 Roofline 与层级信息评估逐层性能，并按数据依赖追踪内存占用，得到峰值内存和整网性能；(6) 生成报告，给出各层与整网性能上界、瓶颈与内存占用，可进一步绘制 batch-size/sequence-length 等分析曲线；(7) 提供 Web 可视化界面，便于配置调整与逐层结果查看。

3. 模型压缩

LLM 的超大参数规模与计算开销给实际部署带来巨大压力，尤其在资源受限场景中更为突出。最直接的缓解路径是模型压缩。本节系统回顾 LLM 的网络压缩方法，覆盖量化、剪枝、知识蒸馏、低秩分解等主流技术。在各小节中，我们结合 LLM-Viewer 对压缩对推理的影响进行分析，并给出部署建议。

3.1. 量化

在 LLM 压缩中，量化是降低存储与计算成本的核心技术。其基本思想是将原始浮点参数映射为整数或离散表示，从而显著减少存储与计算复杂度 [Gholami et al., 2022]。尽管量化会引入一定精度损失，但合理设计可在较小精度代价下获得明显压缩收益。

LLM 量化大体可分两类：(1) 用于压缩预训练模型的量化；(2) 参数高效微调量化 (Q-PEFT)。前者又可分为 QAT 与 PTQ：QAT 在训练或再训练阶段引入量化，使模型适配低比特表示；PTQ 在训练后直接量化，无需再训练，流程更简洁。

3.1.1 LLM-Viewer 用例：量化的 Roofline 分析

这里给出如何用 LLM-Viewer (Sec. 2.3) 分析量化部署瓶颈。在 LLM 中，关键张量包括权重、激活（含临时激活）与 KV cache：(1) 权重必须常驻内存。例如 Llama-13b [Touvron et al., 2023a] 在 FP16 下约占 26GB；(2) 临时激活在推理中产生，例如各层输入在残

³工具开源于 <https://github.com/hahnyuan/LLM-Viewer>

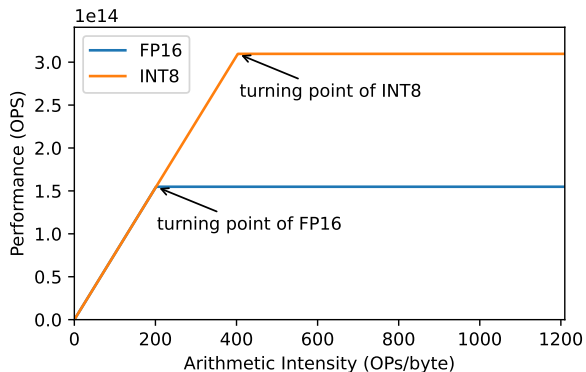


Figure 6. Nvidia A6000 在不同计算数据类型下的 Roofline 示意。

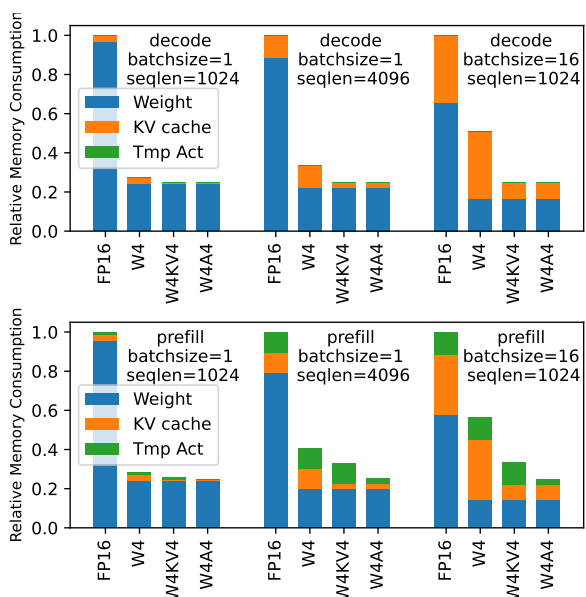


Figure 7. Llama-2-13b 在不同量化设置下的相对内存占用。Tmp Act 表示临时激活。

差加法前需要保留；(3) 自回归模型还需缓存 key/value 以支撑后续 token 生成。我们从计算、内存占用、内存访问三方面分析量化影响。

计算视角：现代硬件（如 NVIDIA GPU）通常支持 FP32/FP16/INT8，且低比特往往有更高算力。例如 A6000 的 FP16 与 INT8 峰值大致为 155TOP/s 与 310TOP/s。在 Roofline 中，启用低比特计算会抬高 compute roof，对 compute-bound 层有利（图 6）。但若仅权重量化为 INT8、激活仍是 FP16，则难以直接利用 INT8 计算单元，往往需要类型转换。同理，当量化比特不受硬件原生支持（如 H100 不支持 INT4 计算）时，也需转换到更高比特执行。

内存占用视角：不同张量量化带来的内存收益不同

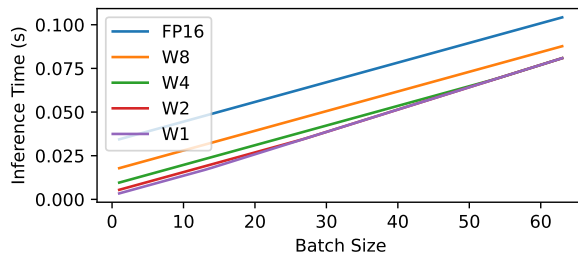


Figure 8. Llama-2-13b decode 阶段在不同量化设置下的推理时间（序列长 = 1024）。

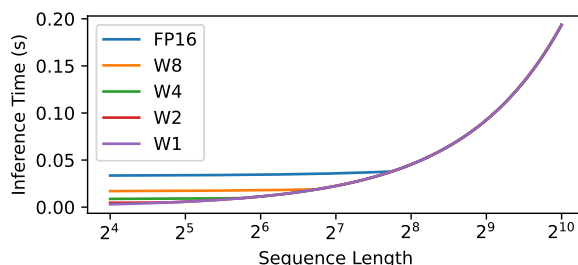


Figure 9. Llama-2-13b prefill 阶段在不同量化设置下的推理时间（batch=1）。

（图 7）⁴。临时激活（特别是 decode 阶段）生命周期短、可快速释放，占比通常较小。而 KV cache 会在完整回答生成结束前持续保留，且随 batch 与序列长度增大而显著增长。

内存访问视角：量化可减少同等计算下的数据搬运字节数，提升算术强度，通常对应三种情况：(1) 量化前后均 memory-bound：访存压力下降，理论性能提升（对 decode 帮助最大）；(2) 从 memory-bound 进入 compute-bound：性能同样提升；(3) 量化前后均 compute-bound：性能提升有限。

图 8 显示：小 batch 下网络前后均 memory-bound，量化可显著降时延；大 batch 下网络已趋于 compute-bound，将权重从 4bit 继续压到 2/1bit 不再明显降时延。图 9 显示：prefill 在短序列时更易 memory-bound，量化有效；序列变长后趋向 compute-bound，收益下降。

3.1.2 用于压缩预训练 LLM 的量化

QAT：QAT [Choi et al., 2018, Courbariaux et al., 2015, Dong et al., 2019] 将量化直接纳入训练过程，使模型适配低精度表示。LLM-QAT [Liu et al., 2023b] 通过数据无关蒸馏缓解训练数据构建问题，并将量化扩展到 KV cache，在吞吐与长序列支持上带来收益。在更低比特场景，Kim et al. [2023b] 提出 TSLD 支持三

⁴W4：仅权重 4bit，激活 FP16；W4A4：权重与激活均 4bit；W4KV4：权重与 KV cache 量化，临时激活仍 FP16。

值 QAT; Shang et al. [2024] 的 PB-LLM 通过“部分二值化”保护关键权重, 维持重度量化下推理能力。

PTQ: PTQ 在训练后量化参数, 不改架构且无需再训练, 实施简单, 尤其适合超大模型。在 LLM 场景中, QAT 成本往往过高, 因此 PTQ 具有较高实践价值。

PTQ 中大量工作聚焦 **仅权重量化**: LUT-GEMM [Park et al., 2023], LLM.int8() [Dettmers et al., 2022], ZeroQuant [Yao et al., 2022], GPTQ [Frantar et al., 2022], AWQ [Kim et al., 2023c, Lin et al., 2023], OWQ [Lee et al., 2023], SpQR [Dettmers et al., 2023b], QuantEase [Behdin et al., 2023] 等, 重点在降低比特并抑制精度损失。更低比特方面, QuIP [Chee et al., 2023], Norm Tweaking Li et al. [2023a], PB-LLM [Shang et al., 2024], BiLLM [Huang et al., 2024] 等推动到 2bit 甚至接近 1bit。

另一类 PTQ 关注 **权重 + 激活量化**: SmoothQuant [Xiao et al., 2023a], RPTQ [Yuan et al., 2023c], OliVe [Guo et al., 2023a], ZeroQuant-FP [Wu et al., 2023d], FPTQ Li et al. [2023c] 等, 主要处理激活离群值、通道差异与硬件友好映射。

自 2023 年底以来, 长上下文快速增长, KV cache 成为关键内存瓶颈。例如 Gemini 1.5 [Sundar Pichai, 2024] 在生产中可处理百万 token; 文档/图像/视频场景也常需万级 token。因此 **KV Cache 量化** 变得尤为重要。2024 年已有多项工作推进该方向, 如 Hooper et al. [2024], KIVI [Liu et al., 2024b], WKVQuant Yue et al. [2024]。图 11 显示, 当序列长度超过 50k, KV cache 占据大部分内存, 其量化可显著降低总内存。

3.1.3 参数高效微调量化 (Q-PEFT)

PEFT 是 LLM 重要方向, 其中 LoRA [Hu et al., 2021, Valipour et al., 2022] 通过低秩分解适配器显著减少可训练参数, 效果接近全量微调, 详见综述 [Hu et al., 2023]。

Q-PEFT 将量化融入 PEFT 流程, 形成新的效率范式。代表工作包括 PEQA [Kim et al., 2023a], DFT [Li et al., 2023e], QLoRA [Dettmers et al., 2023a]。QLoRA 通过新数据类型、双重量化、paged optimizer 等机制显著省内存, 并实现单卡大模型微调。

QLoRA 的限制在于通常难以稳定到 4bit 以下。为此, LQ-LoRA [Guo et al., 2023b], Loft-Q [Li et al., 2023d], QA-LoRA [Xu et al., 2023c] 等工作进一步探索低比特微调, 提升泛化并减少 PTQ 常见精度损失。

3.1.4 LLM 量化讨论

图 10 展示了量化技术演进: 从 PTQ 主导, 逐步过渡到 QAT 与 Q-PEFT 并行发展。这反映了社区针对 PTQ 性能瓶颈的适应性转向, 也表明 QAT/Q-PEFT 正成为高效 LLM 推理的重要增长点。

3.2. 剪枝

剪枝 [LeCun et al., 1989, Liang et al., 2021] 通过删除冗余参数压缩模型, 是另一类主流方法。在 LLM 中, 参数占据主要体量与计算开销, 因此合理剪枝可在较小性能损失下提升效率。剪枝大致分为非结构化与结构化两类。

3.2.1 非结构化剪枝

非结构化剪枝按权重/神经元粒度删除, 通常精度保持较好, 但稀疏模式不规则, 对软件/硬件支持要求高。SparseGPT [Frantar and Alistarh, 2023] 将问题转化为大规模稀疏回归, 单 GPU 数小时可处理 175B 模型, 并在高稀疏率下保持较好性能。Wanda Sun et al. [2023a] 通过权重幅值与输入范数评估重要性, 显著降低重构开销。Yin et al. [2023a] 通过非均匀分层稀疏率重点处理离群层。Flash-LLM [Xia et al., 2023a] 则从硬件侧提出稀疏加载 + 稠密计算以利用 Tensor Core。

3.2.2 结构化剪枝

结构化剪枝删除整层或整块结构, 规则性好、硬件友好, 但对性能影响通常更明显。LLM-Pruner [Ma et al., 2023] 使用一次性结构化剪枝并结合 LoRA 恢复; Sheared Llama [Xia et al., 2023b] 结合目标架构裁剪与动态批加载; Compresso [Guo et al., 2023c] 通过协同学习将 Llama-7B 剪至 5.4B 并维持性能。

3.3. 知识蒸馏

知识蒸馏 [Gou et al., 2021, Hinton et al., 2015] 将大模型 (teacher) 能力迁移给小模型 (student), 在降低资源消耗的同时保持任务能力 [Gou et al., 2021, Shang et al., 2021]。LLM 蒸馏常分为白盒与黑盒两类, 详细综述可见 [Xu et al., 2024]。

3.3.1 白盒蒸馏

白盒蒸馏可访问 teacher 架构与权重, student 除学习输出外还能学习中间表示与决策过程。MiniLLM [Gu et al., 2023], GKD [Agarwal et al., 2023], Homotopic Distillation [Liang et al., 2023a], Liang et al. [2023b], AD-KD [Wu et al., 2023b] 等从损失函数、训练稳定性、层级匹配与可解释归因迁移等角度提升蒸馏质量。

3.3.2 黑盒蒸馏

黑盒蒸馏不依赖 teacher 内部信息, 仅通过输入-输出行为迁移能力。Multitask-ICT [Huang et al., 2022], LaMini-LM [Wu et al., 2023a], PromptMix Sahu et al. [2023], Lion [Jiang et al., 2023] 等是代表性方法。

黑盒蒸馏也常用于迁移 CoT 推理能力。例如 Fu et al. [2023b], SCOTT [Wang et al., 2023], Distilling step-by-step [Hsieh et al., 2023], Li et al. [2023b], Chae

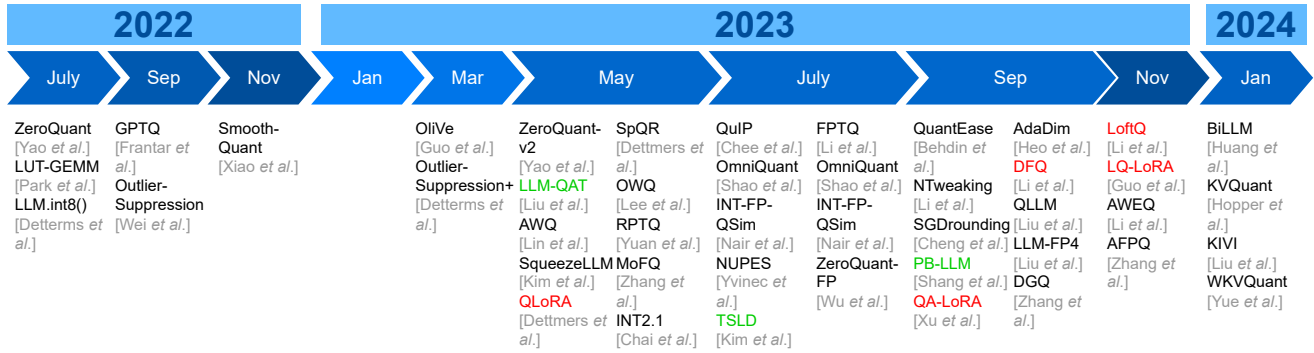


Figure 10. 2022–2024 年 LLM 量化方法时间线。红色为 Q-PEFT，绿色为 QAT，其他为 PTQ。

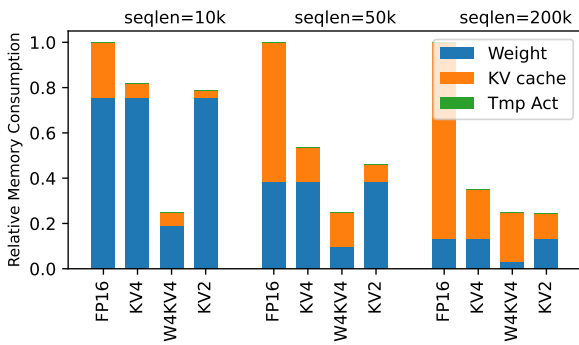


Figure 11. Llama-2-13b decode 阶段在不同量化设置下的内存占用 (batch=1)。

et al. [2023] 等，分别从对比解码监督、多任务蒸馏、符号化 CoT 与对话场景推理蒸馏等角度推进该方向。

3.4. 分解 (Factorization)

低秩分解 [Kishore Kumar and Schneider, 2017] 是压缩 DNN 的经典方法，近年在大模型压缩与加速中持续发展 [Schotthöfer et al., 2022]。

ASVD [Yuan et al., 2023d] 是将分解用于 LLM 压缩的早期代表工作之一，通过激活感知处理离群值并进行分层校准，提高分解精度与效率。LASER [Sharma et al., 2023] 发现选择性移除高阶分量可在若干场景提升模型表现。TensorGPT [Xu et al., 2023b] 则面向 embedding 层，通过 Tensor-Train Decomposition (TTD) [Oseledets, 2011] 以低秩张量形式存储大规模 embedding。

4. 快速解码的算法方法

LLM 在文本生成任务上表现卓越，但其解码阶段通常按自回归方式逐 token 生成。每生成一个 token，都需要重复读取解码器权重。随着模型参数规模增大，解码过程高度 memory-bound [de Jong et al., 2023]，硬件利用率低，导致时延显著增大 [Kim et al., 2023d]。在 ChatBot 等实时交互场景中，这一问题尤为突出，因此

需要专门优化解码过程。

本节从算法角度回顾降低推理成本的方法，主要分两条路径：

- 在固定“每个 token 解码”前提下，如何最小化每次解码所需参数量 (Sec. 4.1)。
- 在固定“每次前向使用参数量”前提下，如何最大化单次前向可解码 token 数 (Sec. 4.2)。

4.1. 每个 token 解码所需最小参数

已有研究指出，即使语言模型参数巨大，生成正确 token 并不总需要全部参数 [Simoulin and Crabbé, 2021]。因此可对输入相关地选择参数子集，从而在保持精度的同时降低时延。本节从三类动态稀疏策略讨论：(1) 早退 (depth/layer 维度动态)；(2) 上下文稀疏 (width 维度动态)；(3) MoE (训练时即构建稀疏专家结构)。

4.1.1 早退 (Early Exiting)

早退思想在编码器模型中已有大量研究 [Baier-Reinio and Sterck, 2020, Hou et al., 2020, Li et al., 2021, Liu et al., 2020, 2022, Schuster et al., 2021, Schwartz et al., 2020, Stickland and Murray, 2019, Xin et al., 2020, Zhou et al., 2020, Zhu, 2021]。但解码器场景更复杂：需要在序列级保持一致性，因为后续 token 依赖先前生成结果。

解码器层结构同构，因此每层隐藏状态都可接 LM Head 产生下一 token 分布。Geva et al. [2022] 与 Simoulin and Crabbé [2021] 观察到：部分 token 在中间层已出现表示饱和，提前退出即可得到与全层一致的 top-1。这构成了解码器早退可行性的基础。

Elbayad et al. [2020] 在机器翻译任务中提出了早期解码器早退框架。其通用范式是：每层后由置信度模块 (固定指标或小 MLP) 评估“当前层是否可饱和退出”，满足条件则提前退出并经 LM Head 输出。

围绕该范式，研究主要集中在四个设计点：

(1) 如何建模“可退出置信度” CALM [Schuster et al., 2022] 对比了 softmax 置信度、层间隐藏态相似度、层内线性分类器三种方案；实验显示线性分类器在“开销-精度”权衡上最佳。[Bae et al., 2023] 发现浅

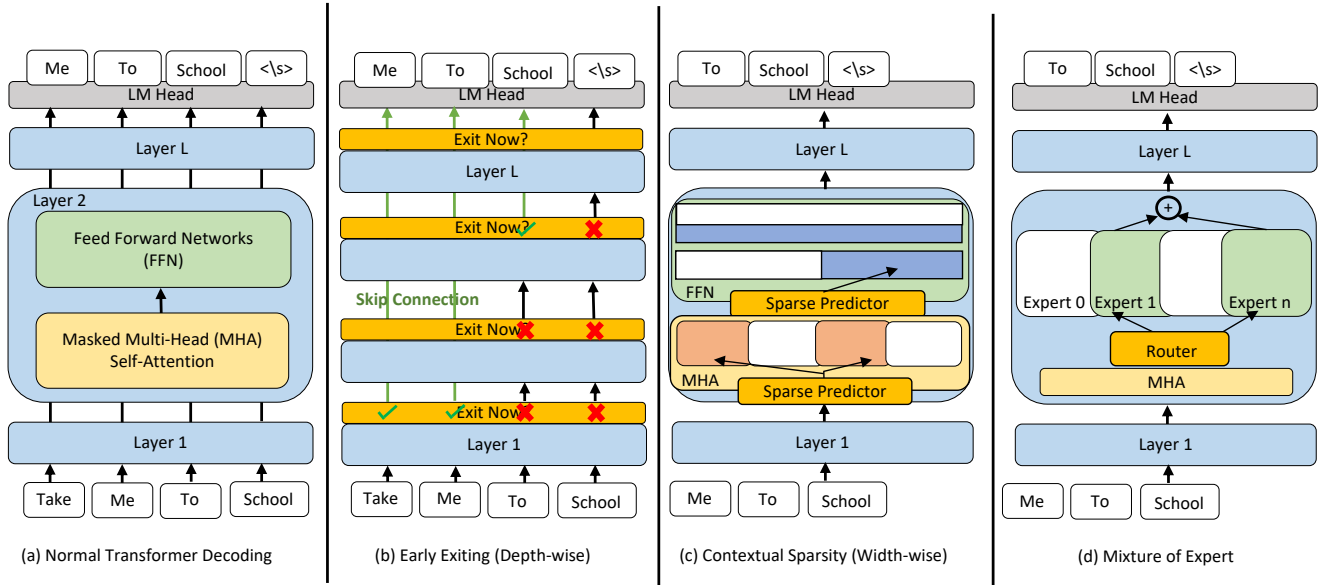


Figure 12. 输入相关动态网络技术示意图

For some tokens, only a necessary subset of weights is needed to decode correctly. (a) presents the normal decoder-only autoregressive model architecture; (b) Early Exiting aims to only use the necessary transformer layers that already produce correct decoded tokens and skip the rest of the layers, but challenges arises when different tokens exit at different layer across. Same gist also applies to the width dimension of the LLM. (c) represents a line of works that predicts the dynamic sparsity on attention head and linear layers column dimension in the input-dependent way. (d) presents the Mixture of Expert method which has the sparsity built-in to the pretrained model. During inference, a router aims to choose concise and proficient expert networks that depending on the input. (Illustration best viewed with color)

层频繁退出会导致输出长度异常，且逐层打分开销高，因此将退出点简化为“浅模块”或“全模型”两选一，减少分类器数量并取得更高加速。ConsistentEE [Zeng et al. \[2023\]](#) 则引入 RL 策略网络，联合优化效率与准确率。

(2) **退出判据如何设定** CALM 通过分布无关校准 (FWER) 给出阈值，且阈值随序列位置指数下降；[Bae et al. \[2023\]](#) 将置信度分布建模为 Beta 分布并在线更新；[Zeng et al. \[2023\]](#) 直接让策略网络输出退出动作。

(3) **隐藏状态传播问题** 当某些 token 提前退出、而后续 token 继续深入层时，后层注意力可能缺失先前 token 的 KV。[Elbayad et al. \[2020\]](#) 与 [Schuster et al. \[2022\]](#) 提出“状态传播”：将早退层隐藏态拷贝到后续缺失层以近似深层状态。后续工作 [Bae et al. \[2023\]](#)，[Ding et al. \[2023\]](#) 发现该近似会损伤性能，并提出在线重算以替代拷贝。[Chen et al. \[2023b\]](#) 通过并行全模型流补齐缺失 KV；[Din et al. \[2023\]](#) 研究用线性桥接层跨层跳跃，平衡精度与开销。SkipDecode [Corro et al. \[2023\]](#) 则采取更激进策略：强制更后位置使用更浅层，避免传播并提升速度。

(4) **输出分类器训练** 中间层退出需要分类器（共享或分层独立）配合。[Elbayad et al. \[2020\]](#) 使用各层 CE 均值训练；[Schuster et al. \[2022\]](#) 对深层赋予更高权重；[Bae et al. \[2023\]](#) 引入动态蒸馏；[Rotem et al. \[2023\]](#) 与 [Ji et al. \[2023\]](#) 指出多目标联合训练会出现梯度冲突并给出缓解策略。此外，[Chen et al. \[2023b\]](#) 还从 3D 并

行角度优化了早退部署。

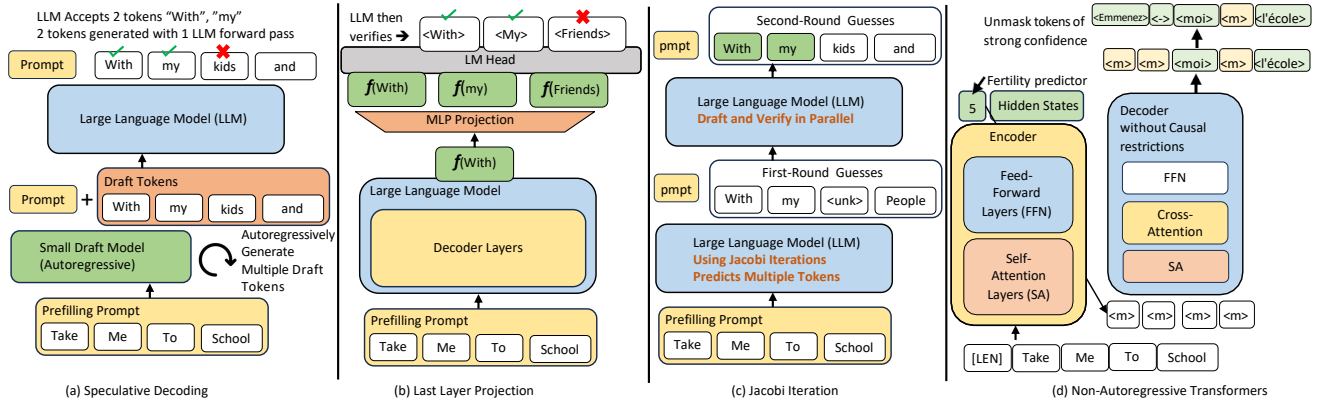
4.1.2 上下文稀疏 (Contextual Sparsity)

若早退对应深度维动态稀疏，上下文稀疏则对应宽度维动态稀疏。Deja Vu [Liu et al. \[2023c\]](#) 系统研究了 LLM 宽度维动态稀疏，报告上下文稀疏率可达 80%，即多数权重可在特定 token 下跳过而保持性能。其核心是为输入 token 动态预测应激活的 attention heads 与 MLP 列，并在 MHA/FFN 前加入小型 Sparse Predictor (图 12(c))，实现超过 2x 推理加速。

PowerInfer [Song et al. \[2023\]](#) 将该思路扩展到 CPU+GPU 异构部署，利用“高频激活神经元常驻 GPU、其余驻留 CPU”的策略配合稀疏预测，提升跨设备推理效率。MatFormer [Devvrit et al. \[2023\]](#) 面向不同硬件能力部署，主要在 FFN 维度做动态结构采样，并通过 Mix’ n’ Match 形成更灵活模型尺寸。

4.1.3 专家混合模型 (MoE)

语言模型在规模上呈现 power-law 增益 [Hoffmann et al. \[2022\]](#)，[Kaplan et al. \[2020\]](#)，但参数膨胀导致训练与推理效率下降。MoE [Yuksel et al. \[2012\]](#) 通过“总参数与每次激活计算解耦”提升效率，并可在不显著增加推理计算的前提下扩展模型能力 [Fedus et al. \[2021\]](#)，[Lepikhin et al. \[2020\]](#)。如图 12(d)，MoE 用专家网络



Several methods have been proposed to generate more than one token per LLM forward prop: (a) shows the speculative decoding which asks the LLM to, instead of generate tokens autoregressively, evaluate tokens generated by the small model all at once to parallelize the memory loading; (b) introduces another approach to have a last layer projection on the large model hidden state vector to generate tokens at subsequent positions; (c) presents another method to treat decoding as solving a system of nonlinear equations and use Jacobi and Gaussian-Seidel iteration algorithm to parallelize the decoding process of a number of token positions; (d) presents the Non-Autoregressive Transformers which targets the seq-to-seq model. The decoder relaxes the autoregressive constraints and iteratively decode all the tokens from masked tokens based on the information from the encoder model hidden states.

Figure 13. 并行解码方法示意图

替换 FFN，并由 gating 为每个 token 选择专家。更系统内容可见稀疏专家综述 Fedus et al. [2022]。

尽管 MoE 与上下文稀疏都依赖输入决定稀疏结构，但二者不同：上下文稀疏是在“已训练稠密模型”上挖掘动态稀疏；MoE 则是从训练阶段即构建稀疏模型。

近期 MoE 成果显著：Sparse Mixer Lee-Thorp and Ainslie [2022]、GLaM Du et al. [2022]、ST-MoE Zoph et al. [2022]、Mixtral 8x7B Jiang et al. [2024] 等在效率与性能间展现强竞争力。部署侧也有优化进展，如 RE-COMPILE Kossmann et al. [2022]、MoE 版 ZeRO 推理 Rajbhandari et al. [2022]、AutoMoE Jawahar et al. [2023]、EdgeMoE Yi et al. [2023] 等。

4.1.4 动态参数缩减的 Roofline 分析

“每 token 最小参数”类方法通常会同时降低计算与访存。从 Roofline 角度看，这类方法往往不会显著改变单算子的算术强度与 bound 类型，但会减少总工作量。

对 Early Exiting / Layer Skipping 而言，整层被跳过，故计算、访存、时延通常近似按比例下降；对 Contextual Sparsity / MoE 而言，不同子算子算术强度差异较大，动态激活模式不同，最终时延收益也会不同。

4.2. 单次前向最大化解码 token 数

另一条降时延路径是放松严格自回归约束，让一次大模型前向尽可能产出多个 token。本节包括两类：(1) speculative decoding (用小草稿模型提议候选，大模型验证)；(2) 直接并行解码 (不依赖额外小 Transformer 草稿模型)。

4.2.1 Speculative Decoding

LLM 解码受权重加载与自回归依赖限制，效率较低。但较小模型常能在“关键 token 被纠正”的前提下逼近大模型序列 Kim et al. [2023e]。因此可让小模型先生成草稿 token，大模型周期性验证/纠正。尽管大模型会额外验证部分错误草稿，FLOPs 可能增加，但权重加载可在 token 维并行摊薄。在 memory-bound 主导下，总体时延仍可显著下降。

分布一致性： Kim et al. [2023e] 早期提出“草稿-回退-回滚”流程（贪心设置）；Leviathan et al. [2023] 与 Chen et al. [2023a] 指出在拒绝位置做重采样可在理论上保持与大模型自回归一致分布。

构建草稿树：草稿序列可接受长度通常有限，越远未来越难猜中。为提高加速比，一类方法在 batch 方向并行多草稿：SpecTr Sun et al. [2023b]、SpecInfer Miao et al. [2023b]、LLMCAD Xu et al. [2023a] 分别从最优传输重采样、树结构构建与边缘部署流水线角度提升验证效率与接受率。

蒸馏与自草稿：提升草稿模型与大模型分布一致性是另一关键。Zhou et al. [2023c] 给出“接受率与模型散度”联系并验证蒸馏收益；Liu et al. [2023a] 进一步提出在线持续蒸馏以适应域漂移。Zhang et al. [2023] 则避免单独草稿模型，通过在大模型内部搜索可跳层子模型作为草稿。

4.2.2 并行解码 (Parallel Decoding)

除 speculative decoding 外，另有大量方法直接让大模型一次前向预测多个未来 token：

多未来 token 直接预测： Stern et al. [2018] 早期在 LM Head 前增加投影层；Medusa Cai et al. [2024] 将该思路扩展到 decoder-only，并配合树结构并行验证；

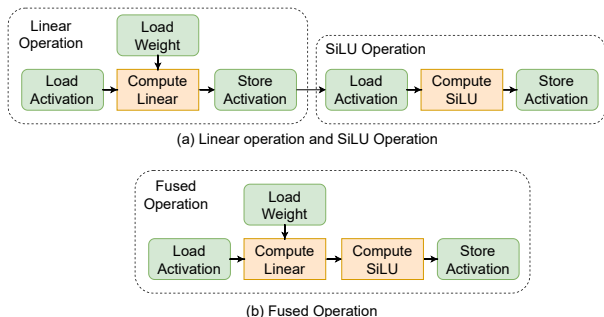


Figure 14. 线性算子与 SiLU 融合示意。

PASS Monea et al. [2023] 使用 lookahead embeddings; EAGLE Li et al. [2024] 通过 FeatExtrapolator 预测未来隐藏态并构建验证树。

高频 n-gram 检索：LLMA Yang et al. [2023] 利用“重复短语”现象进行前缀匹配并由 LLM 验证；REST He et al. [2023] 通过短语库 + trie 检索；Lan et al. [2023] 则增加检索注意力层按相关性选择候选短语。

语言层次结构并行：Skeleton-of-Thought Ning et al. [2023] 先生成提纲再并行扩写要点；APAR Liu et al. [2024a] 结合层次软 token 与 Medusa，进一步提升速度。

Jacobi / Gauss-Seidel 迭代：相关工作表明可用并行迭代近似顺序推理 Ortega and Rheinboldt [2000], Song et al. [2021]。Santilli et al. [2023] 将其用于机器翻译并行解码；Lookahead Decoding Fu et al. [2023a] 将其扩展到 LLM，并结合检索复用与定制 attention mask 提升效率。

非自回归 Transformer (NAT)：NAT 在序列到序列任务上通过并行迭代解码多 token 获得加速，相关研究丰富 Ghazvininejad et al. [2019], Gu et al. [2017], Gu and Kong [2020], Guo et al. [2020], Lee et al. [2018], Li et al. [2019], Savinov et al. [2021], Shao et al. [2020], Sun et al. [2019b], Wang et al. [2019], Wei et al. [2019], 详见综述 Xiao et al. [2023c]。其核心是在 decoder 侧放松自回归约束，用掩码序列迭代去噪生成；但该范式依赖编码器条件，向 decoder-only 体系直接迁移仍有难度。

5. 编译器与系统优化

在模型压缩与算法优化之后，下一步是将 LLM 编译并部署到硬件设备上。为了实现高效推理，可采用多类编译器优化。同时，随着模型规模增大，部署常需多设备协同，形成复杂推理基础设施，系统级优化因此成为热点。本节讨论常见编译/系统优化技术，包括算子融合、内存管理、负载卸载与并行服务。

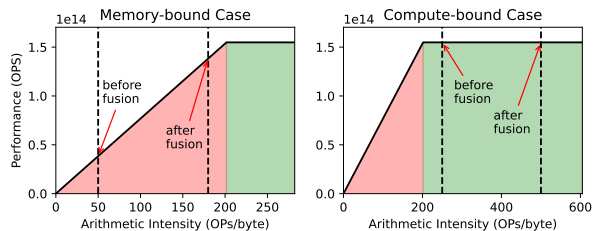


Figure 15. 算子融合在 memory-bound 与 compute-bound 场景下的差异。

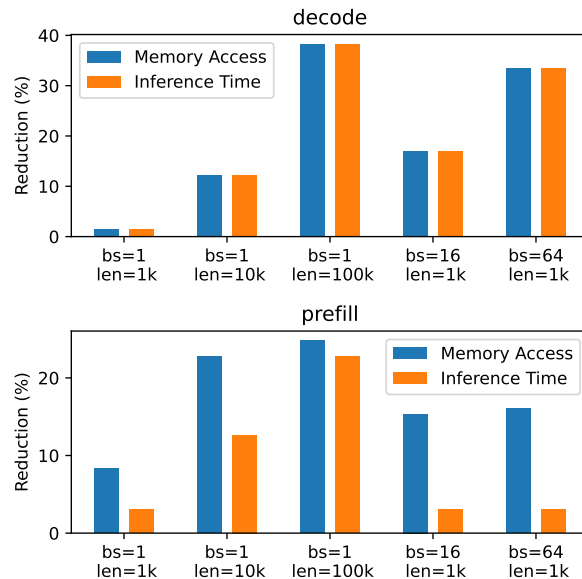


Figure 16. FlashAttention 在 Nvidia A6000 上的访存与时延下降效果。

5.1. 算子融合

算子融合是深度学习编译阶段的重要优化手段。它将计算图中直接相连的多个算子合并执行，从而减少中间结果写回与重复读写。例如“Linear + SiLU”可融合为单核执行（图 14）。这种方式可降低内存占用与访存开销。

Roofline 视角下（图 15），融合通常提升算术强度，对 memory-bound 区域收益明显；若算子已处于 compute-bound 区域，融合收益会较小。

但融合并非总可行：(1) 若中间结果还被其它分支使用，直接融合会引入额外复杂度；(2) 融合后片上缓冲需求可能变大，受限于硬件 on-chip buffer 容量；(3) 框架与硬件实现本身可能对可融合模式有限制。

TVM [Chen et al., 2018] 等编译器可自动识别并执行融合。对 LLM 而言，由于网络结构固定，常更适合采用手工设计的融合模式。典型例子是注意力融合：FlashAttention [Dao, 2023, Dao et al., 2022] 与 Flash-Decoding [Dao et al., 2023] 将注意力中的 MatMul 与

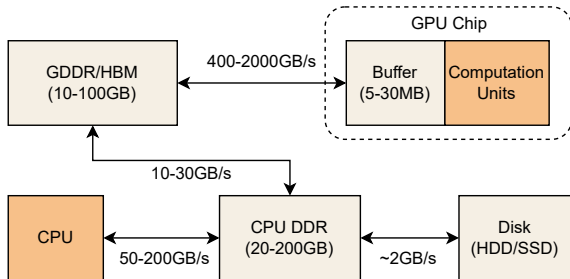


Figure 17. 典型计算机体系中的分层内存空间。

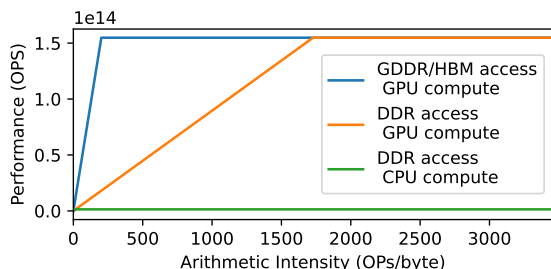


Figure 18. 不同卸载设置下的 Roofline 模型。

softmax 融合，避免显式存取庞大注意力矩阵。图 16 显示其可显著减少访存与时延。值得注意的是 decode 阶段“访存下降约等于时延下降”，而 prefill 阶段时延收益略小于访存收益，原因是 prefill 存在部分 compute-bound 算子。

其它代表性系统包括：DeepSpeed-Inference 的 Deep-Fusion [Aminabadi et al., 2022]、xFormers [Lefaudeux et al., 2022] 的融合 softmax/linear/layer norm/SwiGLU、TensorRT-LLM [Vaidya et al., 2023] 的模式匹配融合。

除了融合，算子内核本身也可进一步优化。例如 FlashDecoding++ [Hong et al., 2023] 通过异步 softmax、flat GEMM 与双缓冲进一步提升效率。

5.2. 内存管理与负载卸载

LLM 推理中输入与输出 token 长度常动态变化：prefill 的序列长度随用户 prompt 变化，decode 随生成逐步增长。这使激活张量形状不同于静态网络，内存管理复杂度上升。

PagedAttention [Kwon et al., 2023] 将 KV cache 分块管理：每条序列的 KV 由多个固定 token 数块组成，并通过映射表把逻辑块映射到 GPU 物理块，机制类似 CPU 虚拟内存。

当 GPU 显存不足以容纳模型时，可采用 workload offloading。如图 17，系统通常含 CPU DDR、GPU GDDR/HBM、磁盘等不同内存层级，带宽差异显著。图 18 表明：把数据存于 CPU 内存并按需传输到 GPU 计算，通常优于直接在 CPU 上计算。当 batch 足够大时，算术强度提升，GPU 更易发挥算力。

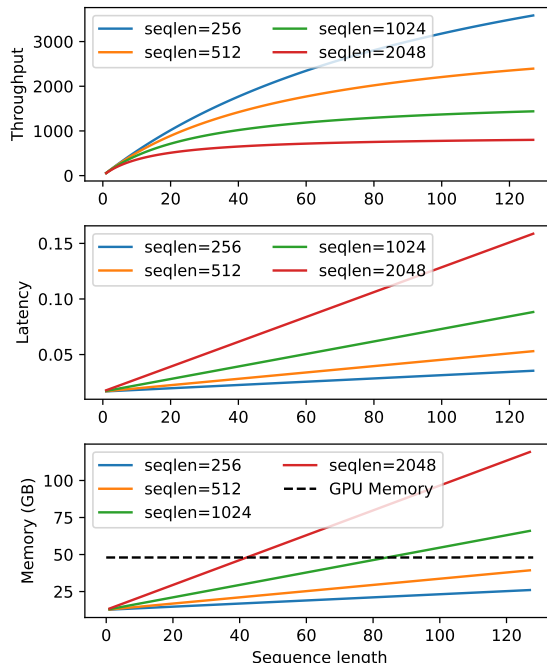


Figure 19. 并行服务策略对 Nvida A6000 (Llama-2-13b) 的吞吐、时延和内存影响。

DeepSpeed-Inference [Aminabadi et al., 2022] 的 ZeRO-Inference 将大模型权重卸载到 CPU，并在大 batch 下通过计算与取数重叠提升效率。HuggingFace Accelerate [HuggingFace, 2022] 支持将部分模块迁移到 CPU/磁盘。FlexGen [Sheng et al., 2023] 在 GPU/CPU/磁盘约束下用线性规划搜索最优吞吐策略。Alizadeh et al. [2023] 则利用闪存大容量特性，将参数驻留闪存并按需搬运到 DRAM。

5.3. 并行服务 (Parallel Serving)

并行服务面向“同时处理多用户请求”。目标一是低延迟（单请求响应快），目标二是高吞吐（单位时间处理更多请求）。系统需在吞吐与延迟之间做平衡。

Batching 是提升吞吐的基础手段。图 19 显示 decode 阶段增大 batch 可显著提升吞吐，但也会增大响应延迟与内存开销。

为优化 batching，已有多项改进：ORCA [Yu et al., 2022] 提出 continuous/rolling batching；SARATHI [Agrawal et al., 2023] 提出 chunked-prefill 与 decode-maximal batching；DeepSpeed-FastGen [Holmes et al., 2024] 与 LightLLM [ModelTC, 2024] 也采用 split-and-fuse 思路提升稳定性与吞吐。

6. 硬件优化

为 LLM 推理设计高效硬件具有挑战性，核心原因在于算术强度⁵在不同阶段与负载配置下差异巨大。

⁵算术强度是“计算量/访存量”比值，见 Roofline 模型 (Sec. 2.2)。

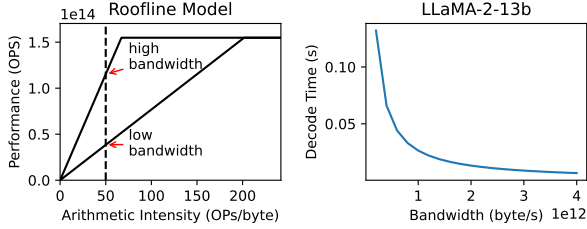


Figure 20. 带宽变化对 Roofline 与 Llama-2-13b (batch=1, seq=1024) 性能的影响。

prefill 阶段通常以 GEMM 批量处理 token，算术强度较高；decode 阶段按 token 逐步生成，常落到 GEMV 或瘦 GEMM，算术强度较低。此外，batch size 与 sequence length 的变化也会显著改变算术强度：大 batch 往往提高算术强度，而长序列会增加 KV cache 读取访存压力。这种“阶段 + 配置双重变化”使硬件设计必须考虑多场景最优。

因此本节围绕“如何应对算术强度变化”回顾硬件优化路径。

6.1. 空间架构 (Spatial Architecture)

LLM 的 decode 自回归特性意味着在大量应用中，时延主要耗在 decode。其低算术强度本质来自对权重与 KV cache 的高频访存。

空间架构提供一种改进思路：与传统“计算单元与主存反复交互”的时分复用方式不同，空间架构将计算分布到多个 PE 并行执行，让中间数据在 PE 间流动，减少回写 DRAM 的频次。

在空间架构下，每个 PE 负责局部计算并与邻近 PE 通信，同时具备独立访存通路。多 PE 并发访问提升系统总带宽，进而提升推理性能。图 20 显示：总带宽提升时，decode 阶段线性层性能可显著增长。

实践方面，Groq 的 LPU [Abts et al., 2022] 构建了面向 LLM 的空间系统，据报告在 Llama-2-70b 上可达每用户 300+ token/s [Groq, 2023]。Graphcore IPU 也是可高效执行 LLM 的空间架构代表 [Graphcore, 2024]。

6.2. 存内计算 (Processing in Memory)

LLM decode 常受“Memory Wall”限制。该问题在体系结构领域长期存在，近年来 PIM（存内/近内计算）成为重要方向：将计算单元放入内存芯片内部，利用更高内部带宽并减少数据搬运。

近年来 DRAM-PIM 开始商用，有望缓解推理带宽瓶颈。表 2 中，UPMEM PIM-DIMM Devaux [2019a] 是早期商用品，采用 DDR4-DIMM 内嵌 RISC 核，但并非面向深度学习设计，峰值带宽与吞吐难满足 LLM。相比之下，Samsung HBM-PIM 将 MAC 集成到 HBM，内部带宽约 2TB/s，且算力约 1.2TFLOPS，更适合 1-2 Ops/Byte 区间算子。

Choi et al. [2023] 提出用 HBM-PIM 加速 batched LLM 中低算术强度 KV-cache 处理，报告 GPU+HBM-

Table 2. Comparison of Commodity DRAM NMC Products

Product	PIM-DIMM	HBM-PIM	AiM
Technique	DDR4	HBM2	GDDR6
PIM Units	RISC Cores	FP16 MAC	BF16 MAC
Peak Bandwidth	80.4 GB/s per DIMM	2 TB/s per cube	1 TB/s per chip
Peak Throughput	43.8 GOP/s per DIMM	1.2 TFLOPS	1 TFLOPS

PIM 系统可达 $3.24\times$ 提速。

SK-hynix 也提出基于 GDDR6 的 AiM Kwon et al. [2022]，采用 BF16 MAC，单芯片约 1TFLOPS 与 1TB/s。尽管尚未报告 LLM 数据，在 LSTM 任务上对比 GPU+HBM2 可达最高 $10\times$ 提升。

尽管 DRAM-PIM 对 memory-intensive 算子潜力显著，仍存在若干限制：

- **计算能力受限。** DRAM 工艺下晶体管速度与逻辑密度劣于同代 CMOS Devaux [2019b]，且金属层少、布线密度低，难集成强算力单元。故更适于小 batch 或 KV 处理，大 batch 高计算场景仍需强主机。
- **容量受限。** PIM 需占用部分存储区域构建计算单元，容量通常较标准内存减少约 50% Kwon et al. [2021]。对需承载大权重与 KV 的 LLM 场景是明显约束。
- **PIM 间通信不足。** PIM 单元分布在各 DRAM bank 附近，跨单元聚合与同步不可避免；但现有 DRAM-PIM 互连能力有限 Jonatan et al. [2024], Zhou et al. [2023d]，常依赖 CPU/GPU 中转，易形成系统低效。

6.3. 新数据格式

神经网络训练通常使用 16/32bit 高精度浮点。高精度 FP 同时兼顾表示精度与范围，但硬件实现复杂，不利于高效推理。

为降低硬件开销，均匀量化将高精度 FP 映射为低精度整数，替代昂贵浮点逻辑。但均匀量化难同时兼顾“范围与精度”，且常需额外算法设计与校准开销来保精度。

非均匀量化尝试通过非均匀分桶与比特分配提升低比特表示精度，但在通用 CPU/GPU 上部署难度更高 [Gholami et al., 2022]。总体上，现有格式难同时满足：高精度、宽范围、高效率、低调参成本。因此大量工作在探索更适配 LLM 的数据格式。

维度	浮点格式	均匀量化	非均匀量化
精度	Good	Bad	Medium
范围	Good	Bad	Medium
硬件效率	Bad	Good	Bad
转换成本	Good	Bad	Medium

Table 3. 浮点、均匀量化与非均匀量化对比

工业界路径之一是从高精度 FP 逐步降低 exponent/mantissa 比特。近期研究 [Micikevicius et al., 2022, Sun et al., 2019a] 表明多类模型（含 LLM）可

从 FP16 直接迁移到 FP8 而精度损失有限。低比特浮点因“高效率 + 低改造成本”受到硬件厂商重视：NVIDIA 在 H100 中集成 FP8 Tensor Core [NVIDIA, 2022], Tesla Dojo 引入可配置 CFloat8 [Tesla, 2023]。

学术界也在推进低比特浮点用于 LLM: ZeroQuant-FP [Wu et al., 2023d] 探索 FP4/FP8; ZeroQuant-(4+2) [Wu et al., 2023c] 与 FP6-LLM [Xia et al., 2024] 探索 FP6 权重量化; LLM-FP4 [Liu et al., 2023b] 将权重与激活均压到 FP4。这些工作证明了更低比特浮点在准确性与效率上的可行性。

另一类研究改进编码方式本身：可变长度编码允许在单值内部动态分配子字段比特，提升“范围-精度”折中能力。代表工作有 ALPS [Langroudi et al., 2021]、ANT [Guo et al., 2022a]、Dybit [Zhou et al., 2023a]。

还有跨值协同编码方向：离群值感知量化 (OLAccel [Park et al., 2018]、GOBO [Zadeh et al., 2020]、OliVe [Guo et al., 2023a]) 对关键大幅值单独处理；bit-sharing (AdaptiveFloat [Tambe et al., 2020]、MX [Darvish Rouhani et al., 2023]) 在块级共享信息，兼顾精度与效率。

方法	描述	收益
低比特浮点	减少指数与尾数比特	Efficiency $\uparrow$
可变长度编码	动态调整子字段位宽	Precision $\uparrow$
离群值感知量化	对离群值定制编码	Precision $\uparrow$
Bit-sharing 编码	块内共享公共信息	Precision $\uparrow$

Table 4. 新数据格式改进方向总结（通常以较小副作用换取收益）

6.4. 新型处理单元 (PE)

除访存瓶颈外，业界也在探索专用处理单元以增强 LLM 关键算子计算效率。这些单元针对 Transformer 负载定制，较通用 CUDA core 等可提供更高性能密度。

NVIDIA 在 H100 中引入 Transformer Engine，可按层统计动态选择 FP16/FP8 执行精度，在精度可控前提下提升性能。也有工作针对注意力机制设计专用加速器 [Kao et al., 2023, Qin et al., 2023]。此外，FPGA 路径（如 DFX [Hong et al., 2022]、FlightLLM [Zeng et al., 2024]）也在持续探索 LLM 推理加速。

7. 讨论

7.1. 可靠性

前文方法可显著提升 LLM 在实践中的训练与推理效率，但压缩与加速也会带来模型可靠性的微妙变化。整体上，Sec. 3 中讨论的压缩方法都会影响可靠性。因此本节聚焦三类关键维度：幻觉（hallucination）、安全对齐（safety alignment）与分布外泛化（OOD generalization）。

幻觉指模型输出与真实知识不一致，产生事实错误或无意义内容 Huang et al. [2023a]。安全对齐关注模型是否能主动识别并拒绝有害请求，避免输出危险内

容 Ouyang et al. [2022]。OOD 可靠性关注模型在长尾或非常规输入（如对抗样本、捷径干扰）下的稳定性 Geirhos et al. [2020]。

7.1.1 幻觉

抑制幻觉能力对参数改动高度敏感。已有研究显示，事实知识常存储在 Transformer 的 FFN 中。因此在量化或结构压缩中，应重点关注 FFN 输出校准。

可行做法是先识别关键 FFN 层，例如利用神经元级解释方法 Meng et al. [2022] 找到对知识存储最关键的层。对这些层在量化时应提高精度配额，即采用“选择性高精度”策略。这样可在整体压缩的同时尽量保持事实知识表示完整，降低幻觉风险。

在剪枝场景，也应尽量保留这些关键 FFN 结构，以维持模型调用事实知识的核心能力。

7.2. 安全对齐

根据既有研究 Yuan et al. [2023c]，中等强度压缩（如 8-bit 量化）通常不会明显削弱模型安全能力。但模型可能对某些 jailbreak 攻击更敏感，而这在既有研究中关注不足 Deng et al. [2023]。因此建议在部署压缩模型前进行系统化 red teaming。此外，基于知识迁移的方案可能显著削弱安全性，建议在迁移后进行额外安全微调。

7.3. OOD 泛化

LLM 在真实部署中常受“决策捷径”影响，导致在长尾子群体上错误率升高 Geirhos et al. [2020]。已有研究 Yuan et al. [2023b] 指出，量化后的网络在同一任务不同子群体间性能差异会显著放大，尤其在依赖上下文捷径的长尾子群体上更明显。此外，实践中常用的 KV-cache 压缩常通过随机丢弃注意力矩阵中的 token 来提效，这可能进一步强化模型捷径依赖。因此建议在下游场景引入对应鲁棒性增强方法，如不变性测试时优化 Ma et al. [2024]。

7.4. 高效大规模多模态模型

7.4.1 大规模多模态模型 (LMM)

大规模多模态模型 (LMM)，尤其视觉语言模型 (VLM)，正成为构建通用智能助手的重要方向，在感知与推理上持续增强。这类模型通常以 LLM 为“认知核心”，并结合多模态基础模型提供高质量表征，实现零样本迁移与上下文学习。其关键挑战在于：如何让 LLM 与其它模态模型高效协同推理。目前主流范式是“多模态预训练 + 多模态指令微调”。更完整讨论可见 [Yin et al., 2023b, Zhang et al., 2024a]。

7.4.2 高效 LMM

在资源受限场景中，多模态能力需求日益明显。尽管 LMM 能力强，但其训练与部署成本高，亟需高效实现。Gemini [Team et al., 2023] 在高效 LMM 方面表现突

出，并提出适配低内存设备的移动级模型，但仍为闭源。开源方向中，LLaVA-v1.5 等已采用 4/8 bit 量化 (bitsandbytes [Dettmers et al., 2022]，详见 Sec. 3)。

其它代表工作包括：MobileVLM [Chu et al., 2023] 与 MobileVLM-v2 [Chu et al., 2024]，TinyGPT-V [Yuan et al., 2023a]，LLaVA-Phi [Zhu et al., 2024]，Varytoy [Wei et al., 2024]，TinyLLaVA [Zhou et al., 2024]，MoE-LLaVA [Lin et al., 2024] 等。这些工作表明在合适架构、数据与训练策略下，小模型多模态能力可逼近大模型。

7.5. 长上下文建模

在 Chatbot、文档摘要等任务中，LLM 长上下文理解与推理能力常成为瓶颈。预训练语料多为短片段文本，难以充分覆盖超长依赖学习。为弥补这一缺口，研究从多个角度推进：本节重点讨论替代注意力 (Sec. 7.5.1)、缓存压缩与检索、位置编码改造。更全面综述可见 Huang et al. [2023b]。

7.5.1 替代注意力设计

自注意力是 Transformer 核心。在 decoder-only + KV cache 推理下，若历史上下文很长，对全部历史 key/value 做注意力会带来高计算与高访存开销。因此大量工作探索“并非所有历史 token 都需被完整关注”。

Landmark Attention Mohtashami and Jaggi [2023] 通过 landmark token 先判断块内信息是否必要；Funnel-Transformer Dai et al. [2020] 通过编码侧下采样与解码侧上采样降低 FLOPs；Longformer Beltagy et al. [2020]、ETC Ainslie et al. [2020]、LongT5 Guo et al. [2022b] 则结合局部窗口与全局摘要机制。

在 LLM 时代，StreamingLLM Xiao et al. [2023b] 与 LM Infinite Han et al. [2023] 提出 sink+sliding window 注意力模式，无需额外微调即可改善长上下文建模。 H_2O Zhang et al. [2024b] 则通过 token 选择减少自注意力复杂度。

7.5.2 递归与检索

Transformer-XL Dai et al. [2019] 提出段级递归，将历史段输出引入当前段，显著延长依赖范围。后续 Segatron Bai et al. [2021] 与 Compressive Transformer Rae et al. [2019] 分别在分段建模与压缩记忆方面扩展。LLM 场景中，Dynamic Memory Compression Nawrot et al. [2024] 继续推进动态压缩历史上下文；Memorizing Transformer Wu et al. [2022]、Memformer Wu et al. [2020] 将检索与局部缓存结合。

另一条路径是 Retrieval-Augmented Generation (RAG)：把远距离历史信息放在二级存储，仅在需要时检索相关 KV/文本片段。RAG 本身体系庞大，综述可见 Gao et al. [2023]，Zhao et al. [2024]。在长上下文实践中，LangChain Pandya and Holia [2023] 是常见方案，通过 embedding 检索缓解长对话记忆问题。其它相关工作还包括 Bertsch et al. [2024]，Borgeaud

et al. [2021]，Guu et al. [2020]，Kynoch et al. [2023]，Modarressi et al. [2023]，Wang et al. [2024b]，Zhong et al. [2023]，Zhou et al. [2023b]。

7.5.3 位置编码改造

预训练期间位置编码通常只见过有限长度。且经典三角函数式位置编码对更长序列外推能力有限。早期方法多在 softmax 前加入注意力偏置：ALiBi Press et al. [2021]、Kerple Chi et al. [2022] 等在长上下文外推中取得效果。

另一条主线聚焦 RoPE 改造：NTK-aware Scaled RoPE Xiong et al. [2023] 调整 RoPE 基参数；LEX Sun et al. [2022]、PermuteFormer Chen [2021] 引入指数衰减；Positional Interpolation 使用线性缩放；Dynamic-NTK Huang and Yau [2019] 动态提高缩放比；YaRN Peng et al. [2023] 对 query/key 线性温度缩放；Giraffe Pal et al. [2023] 用幂律机制保护高频有效信息。

8. 结论

本文系统回顾了高效大语言模型 (LLM) 推理。针对这一实践驱动问题，我们不仅总结了已有研究，还构建了面向部署分析的 Roofline 框架。我们首先通过 Roofline 模型定位 LLM 部署瓶颈，帮助研究者选择更有针对性的优化策略。随后，综述覆盖了该领域关键方向的最新进展，包括模型压缩、解码算法改进、系统级优化与硬件级优化。本项目将持续更新与维护。

References

- Abts, D., Kimmell, G., Ling, A., Kim, J., Boyd, M., Bitar, A., Parmar, S., Ahmed, I., DiCecco, R., Han, D., et al. (2022). A software-defined tensor streaming multiprocessor for large-scale machine learning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 567–580. [12](#)
- Agarwal, R., Vieillard, N., Stanczyk, P., Ramos, S., Geist, M., and Bachem, O. (2023). Gkd: Generalized knowledge distillation for auto-regressive sequence models. *arXiv preprint arXiv:2306.13649*. [6](#)
- Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., and Ramjee, R. (2023). Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*. [11](#)
- Ainslie, J., Ontanon, S., Alberti, C., Cvicek, V., Fisher, Z., Pham, P., Ravula, A., Sanghai, S., Wang, Q., and Yang, L. (2020). Etc: Encoding long and structured inputs in transformers. *arXiv preprint arXiv:2004.08483*. [14](#)
- Alizadeh, K., Mirzadeh, I., Belenko, D., Khatamifard, K., Cho, M., Del Mundo, C. C., Rastegari, M., and Farahtabar, M. (2023). Llm in a flash: Efficient large language model inference with limited memory. *arXiv preprint arXiv:2312.11514*. [11](#)
- Aminabadi, R. Y., Rajbhandari, S., Awan, A. A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Zhang, M., Rasley, J., et al. (2022). Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE. [11](#)
- Bae, S., Ko, J., Song, H., and Yun, S.-Y. (2023). Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. [7](#), [8](#)
- Bai, H., Shi, P., Lin, J., Xie, Y., Tan, L., Xiong, K., Gao, W., and Li, M. (2021). Segatron: Segment-aware transformer for language modeling and understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12526–12534. [14](#)
- Baier-Reinio, A. and Sterck, H. D. (2020). N-node transformer: A depth-adaptive variant of the transformer using neural ordinary differential equations. [7](#)
- Behdin, K., Acharya, A., Gupta, A., Keerthi, S., and Mazumder, R. (2023). Quantease: Optimization-based quantization for language models—an efficient and intuitive algorithm. *arXiv preprint arXiv:2309.01885*. [6](#)
- Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*. [14](#)
- Bertsch, A., Alon, U., Neubig, G., and Gormley, M. (2024). Unlimiformer: Long-range transformers with unlimited length input. *Advances in Neural Information Processing Systems*, 36. [14](#)
- Borgeaud, S. et al. (2021). Improving language models by retrieving from trillions of tokens. arxiv e-prints, art. *arXiv preprint arXiv:2112.04426*. [14](#)
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901. [1](#)
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. (2024). Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*. [9](#)
- Chae, H., Song, Y., Ong, K. T.-i., Kwon, T., Kim, M., Yu, Y., Lee, D., Kang, D., and Yeo, J. (2023). Dialogue chain-of-thought distillation for commonsense-aware conversational agents. *arXiv preprint arXiv:2310.09343*. [6](#)
- Chee, J., Cai, Y., Kuleshov, V., and De Sa, C. (2023). Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*. [6](#)
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. (2023a). Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*. [9](#)
- Chen, P. (2021). Permuteformer: Efficient relative position encoding for long sequences. [14](#)
- Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., et al. (2018). Tvm: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594. [10](#)
- Chen, Y., Pan, X., Li, Y., Ding, B., and Zhou, J. (2023b). Ee-llm: Large-scale training and inference of early-exit large language models with 3d parallelism. [8](#)
- Chi, T.-C., Fan, T.-H., Ramadge, P. J., and Rudnicky, A. (2022). Kerple: Kernelized relative positional embedding for length extrapolation. *Advances in Neural Information Processing Systems*, 35:8386–8399. [14](#)
- Choi, J., Park, J., Kyung, K., Kim, N. S., and Ahn, J. H. (2023). Unleashing the potential of pim: Accelerating large batched inference of transformer-based generative models. *IEEE Computer Architecture Letters*. [12](#)
- Choi, J., Wang, Z., Venkataramani, S., Chuang, P. I.-J., Srinivasan, V., and Gopalakrishnan, K. (2018). Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*. [5](#)

- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., Schuh, P., Shi, K., Tsvyashchenko, S., Maynez, J., Rao, A., Barnes, P., Tay, Y., Shazeer, N., Prabhakaran, V., Reif, E., Du, N., Hutchinson, B., Pope, R., Bradbury, J., Austin, J., Isard, M., Gur-Ari, G., Yin, P., Duke, T., Levskaya, A., Ghemawat, S., Dev, S., Michalewski, H., Garcia, X., Misra, V., Robinson, K., Fedus, L., Zhou, D., Ippolito, D., Luan, D., Lim, H., Zoph, B., Spiridonov, A., Sepassi, R., Dohan, D., Agrawal, S., Omerick, M., Dai, A. M., Pillai, T. S., Pel-lat, M., Lewkowycz, A., Moreira, E., Child, R., Polozov, O., Lee, K., Zhou, Z., Wang, X., Saeta, B., Diaz, M., Firat, O., Catasta, M., Wei, J., Meier-Hellstern, K., Eck, D., Dean, J., Petrov, S., and Fiedel, N. (2022). Palm: Scaling language modeling with pathways.
- Chu, X., Qiao, L., Lin, X., Xu, S., Yang, Y., Hu, Y., Wei, F., Zhang, X., Zhang, B., Wei, X., et al. (2023). Mobilevlm: A fast, reproducible and strong vision language assistant for mobile devices. *arXiv preprint arXiv:2312.16886*. 14
- Chu, X., Qiao, L., Zhang, X., Xu, S., Wei, F., Yang, Y., Sun, X., Hu, Y., Lin, X., Zhang, B., et al. (2024). Mobilevlm v2: Faster and stronger baseline for vision language model. *arXiv preprint arXiv:2402.03766*. 14
- Corro, L. D., Giorno, A. D., Agarwal, S., Yu, B., Awadallah, A., and Mukherjee, S. (2023). Skipdecode: Autoregressive skip decoding with batching and caching for efficient llm inference. 8
- Courbariaux, M., Bengio, Y., and David, J.-P. (2015). Binaryconnect: Training deep neural networks with binary weights during propagations. *Advances in neural information processing systems*, 28. 5
- Dai, Z., Lai, G., Yang, Y., and Le, Q. (2020). Funnel-transformer: Filtering out sequential redundancy for efficient language processing. *Advances in neural information processing systems*, 33:4271–4282. 14
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. (2019). Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*. 14
- Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning. 10
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359. 10
- Dao, T., Haziza, D., Massa, F., and Sizov, G. (2023). Flash-decoding for long-context inference. 10
- Darvish Rouhani, B., Zhao, R., Elango, V., Shafipour, R., Hall, M., Mesmakhosroshahi, M., More, A., Melnick, L., Golub, M., Varatkar, G., et al. (2023). With shared microexponents, a little shifting goes a long way. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–13. 13
- de Jong, M., Zemlyanskiy, Y., Ainslie, J., FitzGerald, N., Sanghai, S., Sha, F., and Cohen, W. (2023). Fido: Fusion-in-decoder optimized for stronger performance and faster inference. 7
- Deng, G., Liu, Y., Li, Y., Wang, K., Zhang, Y., Li, Z., Wang, H., Zhang, T., and Liu, Y. (2023). Jail-breaker: Automated jailbreak across multiple large language model chatbots. *arXiv preprint arXiv:2307.08715*. 13
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*. 6, 14
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023a). Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*. 6
- Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoeffler, T., and Alistarh, D. (2023b). Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*. 6
- Dettmers, T. and Zettlemoyer, L. (2023). The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning*, pages 7750–7774. PMLR.
- Devaux, F. (2019a). The true processing in memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–24. IEEE Computer Society. 12
- Devaux, F. (2019b). The true processing in memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–24. IEEE Computer Society. 12
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding.
- Devvrit, Kudugunta, S., Kusupati, A., Dettmers, T., Chen, K., Dhillon, I., Tsvetkov, Y., Hajishirzi, H., Kakade, S., Farhadi, A., and Jain, P. (2023). Matformer: Nested transformer for elastic inference. 8
- Din, A. Y., Karidi, T., Choshen, L., and Geva, M. (2023). Jump to conclusions: Short-cutting transformers with linear transformations. 8
- Ding, T., Chen, T., Zhu, H., Jiang, J., Zhong, Y., Zhou, J., Wang, G., Zhu, Z., Zharkov, I., and Liang, L. (2023). The efficiency spectrum of large language models: An algorithmic survey. *arXiv preprint arXiv:2312.00678*. 1, 8

- Dong, Z., Yao, Z., Gholami, A., Mahoney, M. W., and Keutzer, K. (2019). Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 293–302. 5
- Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., et al. (2022). Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*, pages 5547–5569. PMLR. 9
- Elbayad, M., Gu, J., Grave, E., and Auli, M. (2020). Depth-adaptive transformer. 7, 8
- Fan, A., Lavril, T., Grave, E., Joulin, A., and Sukhbaatar, S. (2020). Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*.
- Fedus, W., Dean, J., and Zoph, B. (2022). A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667*. 9
- Fedus, W., Zoph, B., and Shazeer, N. (2021). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.(2021). *arXiv preprint cs.LG/2101.03961*. 8
- Frantar, E. and Alistarh, D. (2023). Sparsegpt: Massive language models can be accurately pruned in one-shot. *ICML*. 6
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2022). Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*. 6
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. (2023a). Breaking the sequential dependency of llm inference using lookahead decoding. 10
- Fu, Y., Peng, H., Ou, L., Sabharwal, A., and Khot, T. (2023b). Specializing smaller language models towards multi-step reasoning. *arXiv preprint arXiv:2301.12726*. 6
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. 14
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673. 13
- Geva, M., Caciularu, A., Wang, K. R., and Goldberg, Y. (2022). Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. 7
- Ghazvininejad, M., Levy, O., Liu, Y., and Zettlemoyer, L. (2019). Mask-predict: Parallel decoding of conditional masked language models. 10
- Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., and Keutzer, K. (2022). A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, pages 291–326. Chapman and Hall/CRC. 4, 12
- Gou, J., Yu, B., Maybank, S. J., and Tao, D. (2021). Knowledge distillation: A survey. *International Journal of Computer Vision*, 129:1789–1819. 6
- Graphcore (2024). Tasks and tutorials using graphcore’s ipu with hugging face. <https://github.com/graphcore/Gradient-HuggingFace>. Accessed on March 10, 2024. 12
- Groq (2023). Groq sets new large language model performance record of 300 tokens per second per user on meta ai foundational llm llama 2.70b. <https://www.groq.com/groq-sets-new-large-language-model-performance-record-of-300-tokens-per-second-per-user-on-meta-ai-foundational-llm-llama-2-70b/>. 12
- Gu, J., Bradbury, J., Xiong, C., Li, V. O., and Socher, R. (2017). Non-autoregressive neural machine translation. *arXiv preprint arXiv:1711.02281*. 10
- Gu, J. and Kong, X. (2020). Fully non-autoregressive neural machine translation: Tricks of the trade. 10
- Gu, Y., Dong, L., Wei, F., and Huang, M. (2023). Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*. 6
- Guo, C., Tang, J., Hu, W., Leng, J., Zhang, C., Yang, F., Liu, Y., Guo, M., and Zhu, Y. (2023a). Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15. 6, 13
- Guo, C., Zhang, C., Leng, J., Liu, Z., Yang, F., Liu, Y., Guo, M., and Zhu, Y. (2022a). Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1414–1433. IEEE. 13
- Guo, H., Greengard, P., Xing, E. P., and Kim, Y. (2023b). Lq-lora: Low-rank plus quantized matrix decomposition for efficient language model finetuning. *arXiv preprint arXiv:2311.12023*. 6
- Guo, J., Xu, L., and Chen, E. (2020). Jointly masked sequence-to-sequence model for non-autoregressive neural machine translation. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 376–385, Online. Association for Computational Linguistics. 10

- Guo, M., Ainslie, J., Uthus, D., Ontanon, S., Ni, J., Sung, Y.-H., and Yang, Y. (2022b). Longt5: Efficient text-to-text transformer for long sequences. [14](#)
- Guo, S., Xu, J., Zhang, L. L., and Yang, M. (2023c). Compresso: Structured pruning with collaborative prompting learns compact large language models. *arXiv preprint arXiv:2310.05015*. [6](#)
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M. (2020). Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR. [14](#)
- Han, C., Wang, Q., Xiong, W., Chen, Y., Ji, H., and Wang, S. (2023). Lm-infinite: Simple on-the-fly length generalization for large language models. *arXiv preprint arXiv:2308.16137*. [14](#)
- He, Z., Zhong, Z., Cai, T., Lee, J. D., and He, D. (2023). Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*. [10](#)
- Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*. [6](#)
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. (2022). Training compute-optimal large language models. [8](#)
- Holmes, C., Tanaka, M., Wyatt, M., Awan, A. A., Rasley, J., Rajbhandari, S., Aminabadi, R. Y., Qin, H., Bakhtiari, A., Kurilenko, L., et al. (2024). Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671*. [11](#)
- Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, H., and Wang, Y. (2023). Flashdecoding++: Faster large language model inference on gpus. *arXiv preprint arXiv:2311.01282*. [11](#)
- Hong, S., Moon, S., Kim, J., Lee, S., Kim, M., Lee, D., and Kim, J.-Y. (2022). Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 616–630. IEEE. [13](#)
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. (2024). Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*. [6](#)
- Hou, L., Huang, Z., Shang, L., Jiang, X., Chen, X., and Liu, Q. (2020). Dynabert: Dynamic bert with adaptive width and depth. [7](#)
- Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*. [6](#)
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*. [6](#)
- Hu, Z., Lan, Y., Wang, L., Xu, W., Lim, E.-P., Lee, R. K.-W., Bing, L., and Poria, S. (2023). Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. *arXiv preprint arXiv:2304.01933*. [6](#)
- Huang, J. and Yau, H.-T. (2019). Dynamics of deep neural networks and neural tangent hierarchy. [14](#)
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., et al. (2023a). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*. [13](#)
- Huang, W., Liu, Y., Qin, H., Li, Y., Zhang, S., Liu, X., Magno, M., and Qi, X. (2024). Billm: Pushing the limit of post-training quantization for llms. *arXiv preprint arXiv:2402.04291*. [6](#)
- Huang, Y., Chen, Y., Yu, Z., and McKeown, K. (2022). In-context learning distillation: Transferring few-shot learning ability of pre-trained language models. *arXiv preprint arXiv:2212.10670*. [6](#)
- Huang, Y., Xu, J., Jiang, Z., Lai, J., Li, Z., Yao, Y., Chen, T., Yang, L., Xin, Z., and Ma, X. (2023b). Advancing transformer architecture in long-context large language models: A comprehensive survey. *arXiv preprint arXiv:2311.12351*. [14](#)
- Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., and Bengio, Y. (2016). Binarized neural networks. *Advances in neural information processing systems*, 29.
- HuggingFace (2022). Hugging face accelerate. [11](#)
- Javaheripi, M., Bubeck, S., Abdin, M., Aneja, J., Bubeck, S., Mendes, C. C. T., Chen, W., Del Giorno, A., Eldan, R., Gopi, S., et al. (2023). Phi-2: The surprising power of small language models. *Microsoft Research Blog*.
- Jawahar, G., Mukherjee, S., Liu, X., Kim, Y. J., Mageed, M. A., Laks Lakshmanan, V., Hassan, A., Bubeck, S., and Gao, J. (2023). Automoe: Heterogeneous mixture-of-experts with adaptive computation for efficient neural machine translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 9116–9132. [9](#)

- Ji, Y., Wang, J., Li, J., Chen, Q., Chen, W., and Zhang, M. (2023). Early exit with disentangled representation and equiangular tight frame. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 14128–14142, Toronto, Canada. Association for Computational Linguistics. 8
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. (2024). Mixtral of experts. *arXiv preprint arXiv:2401.04088*. 9
- Jiang, Y., Chan, C., Chen, M., and Wang, W. (2023). Lion: Adversarial distillation of closed-source large language model. *arXiv preprint arXiv:2305.12870*. 6
- Jonatan, G., Cho, H., Son, H., Wu, X., Livesay, N., Mora, E., Shivdikar, K., Abellán, J. L., Joshi, A., Kaeli, D., et al. (2024). Scalability limitations of processing-in-memory using real system evaluations. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(1):1–28. 12
- Kao, S.-C., Subramanian, S., Agrawal, G., Yazdanbakhsh, A., and Krishna, T. (2023). Flat: An optimized dataflow for mitigating attention bottlenecks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 295–310. 13
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. 8
- Kim, J., Lee, J. H., Kim, S., Park, J., Yoo, K. M., Kwon, S. J., and Lee, D. (2023a). Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. *arXiv preprint arXiv:2305.14152*. 6
- Kim, M., Lee, S., Lee, J., Hong, S., Chang, D.-S., Sung, W., and Choi, J. (2023b). Token-scaled logit distillation for ternary weight generative language models. *arXiv preprint arXiv:2308.06744*. 5
- Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. (2023c). Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*. 6
- Kim, S., Hooper, C., Wattanawong, T., Kang, M., Yan, R., Genc, H., Dinh, G., Huang, Q., Keutzer, K., Mahoney, M. W., Shao, Y. S., and Gholami, A. (2023d). Full stack optimization of transformer inference: a survey. 7
- Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. (2023e). Speculative decoding with big little decoder. 9
- Kishore Kumar, N. and Schneider, J. (2017). Literature survey on low rank approximation of matrices. *Linear and Multilinear Algebra*, 65(11):2212–2244. 7
- Kossmann, F., Jia, Z., and Aiken, A. (2022). Optimizing mixture of experts using dynamic recompilations. *arXiv preprint arXiv:2205.01848*. 9
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. 11
- Kwon, Y., Lee, S. H., Lee, J., Kwon, S., Ryu, J., Son, J., O, S., Yu, H., Lee, H., Kim, S. Y., Cho, Y., Kim, J. G., Choi, J., Shin, H., Kim, J., Phuach, B., Kim, H., Song, M. J., Choi, A., Kim, D., Kim, S., Kim, E., Wang, D., Kang, S., Ro, Y., Seo, S., Song, J., Youn, J., Sohn, K., and Kim, N. S. (2021). 25.4 A 20nm 6gb function-in-memory dram, based on HBM2 with a 1.2tflops programmable computing unit using bank-level parallelism, for machine learning applications. In *IEEE International Solid-State Circuits Conference, ISSCC 2021, San Francisco, CA, USA, February 13-22, 2021*, pages 350–352. IEEE. 12
- Kwon, Y., Vladimir, K., Kim, N., Shin, W., Won, J., Lee, M., Joo, H., Choi, H., Kim, G., An, B., et al. (2022). System architecture and software stack for gddr6-aim. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, pages 1–25. IEEE. 12
- Kynoch, B., Latapie, H., and van der Sluis, D. (2023). Recallm: An adaptable memory mechanism with temporal understanding for large language models. 14
- Lan, T., Cai, D., Wang, Y., Huang, H., and Mao, X.-L. (2023). Copy is all you need. 10
- Langroudi, H. F., Karia, V., Carmichael, Z., Ziyarah, A., Pandit, T., Gustafson, J. L., and Kudithipudi, D. (2021). Alps: Adaptive quantization of deep neural networks with generalized posits. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3100–3109. 13
- LeCun, Y., Denker, J., and Solla, S. (1989). Optimal brain damage. *Advances in neural information processing systems*, 2. 6
- Lee, C., Jin, J., Kim, T., Kim, H., and Park, E. (2023). Owq: Lessons learned from activation outliers for weight quantization in large language models. *arXiv preprint arXiv:2306.02272*. 6
- Lee, J., Mansimov, E., and Cho, K. (2018). Deterministic non-autoregressive neural sequence modeling by iterative refinement. 10

- Lee-Thorp, J. and Ainslie, J. (2022). Sparse mixers: Combining moe and mixing to build a more efficient bert. *arXiv preprint arXiv:2205.12399*. 9
- Lefaudeux, B., Massa, F., Liskovich, D., Xiong, W., Caggiano, V., Naren, S., Xu, M., Hu, J., Tintore, M., Zhang, S., Labatut, P., Haziza, D., Wehrstedt, L., Reizenstein, J., and Sizov, G. (2022). xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>. 11
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. (2020). Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*. 8
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. 9
- Li, L., Li, Q., Zhang, B., and Chu, X. (2023a). Norm tweaking: High-performance low-bit quantization of large language models. *arXiv preprint arXiv:2309.02784*. 6
- Li, L., Lin, Y., Chen, D., Ren, S., Li, P., Zhou, J., and Sun, X. (2021). Cascadebert: Accelerating inference of pre-trained language models via calibrated complete models cascade. 7
- Li, L. H., Hessel, J., Yu, Y., Ren, X., Chang, K.-W., and Choi, Y. (2023b). Symbolic chain-of-thought distillation: Small models can also “think” step-by-step. *arXiv preprint arXiv:2306.14050*. 6
- Li, Q., Zhang, Y., Li, L., Yao, P., Zhang, B., Chu, X., Sun, Y., Du, L., and Xie, Y. (2023c). Fptq: Fine-grained post-training quantization for large language models. *arXiv preprint arXiv:2308.15987*. 6
- Li, Y., Wei, F., Zhang, C., and Zhang, H. (2024). Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*. 10
- Li, Y., Yu, Y., Liang, C., He, P., Karampatziakis, N., Chen, W., and Zhao, T. (2023d). Loftq: Lora-fine-tuning-aware quantization for large language models. *arXiv preprint arXiv:2310.08659*. 6
- Li, Z., Lin, Z., He, D., Tian, F., Qin, T., Wang, L., and Liu, T.-Y. (2019). Hint-based training for non-autoregressive machine translation. *arXiv preprint arXiv:1909.06708*. 10
- Li, Z., Liu, X., Zhu, B., Dong, Z., Gu, Q., and Keutzer, K. (2023e). Qft: Quantized full-parameter tuning of llms with affordable resources. *arXiv preprint arXiv:2310.07147*. 6
- Liang, C., Jiang, H., Li, Z., Tang, X., Yin, B., and Zhao, T. (2023a). Homodistil: Homotopic task-agnostic distillation of pre-trained transformers. *arXiv preprint arXiv:2302.09632*. 6
- Liang, C., Zuo, S., Zhang, Q., He, P., Chen, W., and Zhao, T. (2023b). Less is more: Task-aware layer-wise distillation for language model compression. In *International Conference on Machine Learning*, pages 20852–20867. PMLR. 6
- Liang, T., Glossner, J., Wang, L., Shi, S., and Zhang, X. (2021). Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing*, 461:370–403. 6
- Lin, B., Tang, Z., Ye, Y., Cui, J., Zhu, B., Jin, P., Zhang, J., Ning, M., and Yuan, L. (2024). Moe-llava: Mixture of experts for large vision-language models. *arXiv preprint arXiv:2401.15947*. 14
- Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., and Han, S. (2023). Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*. 6
- Liu, M., Zeng, A., Wang, B., Zhang, P., Tang, J., and Dong, Y. (2024a). Apar: Llms can do auto-parallel autoregressive decoding. 10
- Liu, W., Zhou, P., Zhao, Z., Wang, Z., Deng, H., and Ju, Q. (2020). Fastbert: a self-distilling bert with adaptive inference time. 7
- Liu, X., Hu, L., Bailis, P., Stoica, I., Deng, Z., Cheung, A., and Zhang, H. (2023a). Online speculative decoding. *arXiv preprint arXiv:2310.07177*. 9
- Liu, X., Sun, T., He, J., Wu, J., Wu, L., Zhang, X., Jiang, H., Cao, Z., Huang, X., and Qiu, X. (2022). Towards efficient NLP: A standard evaluation and a strong baseline. In Carpuat, M., de Marneffe, M.-C., and Meza Ruiz, I. V., editors, *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3288–3303, Seattle, United States. Association for Computational Linguistics. 7
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., and Chandra, V. (2023b). Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*. 5, 13
- Liu, Z., Wang, J., Dao, T., Zhou, T., Yuan, B., Song, Z., Shrivastava, A., Zhang, C., Tian, Y., Re, C., and Chen, B. (2023c). Deja vu: Contextual sparsity for efficient llms at inference time. 8
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. (2024b). Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*. 6
- Ma, H., Zhu, Y., Zhang, C., Zhao, P., Wu, B., Huang, L.-K., Hu, Q., and Wu, B. (2024). Invariant test-time adaptation for vision-language model generalization. *arXiv preprint arXiv:2403.00376*. 13

- Ma, X., Fang, G., and Wang, X. (2023). Llm-pruner: On the structural pruning of large language models. *arXiv preprint arXiv:2305.11627*. 6
- Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2022). Locating and editing factual associations in GPT. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. 13
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Jin, H., Chen, T., and Jia, Z. (2023a). Towards efficient generative large language model serving: A survey from algorithms to systems. *arXiv preprint arXiv:2312.15234*. 1
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Wong, R. Y. Y., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. (2023b). Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*. 9
- Mickevicus, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., et al. (2022). Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*. 12
- Modarressi, A., Imani, A., Fayyaz, M., and Schütze, H. (2023). Ret-llm: Towards a general read-write memory for large language models. 14
- ModelTC (2024). Lightllm: A python-based llm inference and serving framework. <https://github.com/ModelTC/lightllm>. 11
- Mohtashami, A. and Jaggi, M. (2023). Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*. 14
- Monea, G., Joulin, A., and Grave, E. (2023). Pass: Parallel speculative sampling. *arXiv preprint arXiv:2311.13581*. 10
- Nawrot, P., Łańcucki, A., Chochowski, M., Tarjan, D., and Ponti, E. M. (2024). Dynamic memory compression: Retrofitting llms for accelerated inference. *arXiv preprint arXiv:2403.09636*. 14
- Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., and Wang, Y. (2023). Skeleton-of-thought: Large language models can do parallel decoding. 10
- NVIDIA (2022). Inside the nvidia hopper architecture. <https://www.nvidia.com/en-us/data-center/technologies/hopper-architecture/>. Accessed on March 11, 2024. 13
- Ortega, J. M. and Rheinboldt, W. C. (2000). *Iterative solution of nonlinear equations in several variables*. SIAM. 10
- Oseledets, I. V. (2011). Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317. 7
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744. 1, 13
- Pal, A., Karkhanis, D., Roberts, M., Dooley, S., Sundararajan, A., and Naidu, S. (2023). Giraffe: Adventures in expanding context lengths in llms. 14
- Pandya, K. and Holia, M. (2023). Automating customer service using langchain: Building custom open-source gpt chatbot for organizations. *arXiv preprint arXiv:2310.05421*. 14
- Park, E. et al. (2018). Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *ISCA*, pages 688–698. IEEE. 13
- Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. (2023). Lutgemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models. *arXiv preprint arXiv:2206.09557*. 6
- Peng, B., Quesnelle, J., Fan, H., and Shippole, E. (2023). Yarn: Efficient context window extension of large language models. 14
- Press, O., Smith, N. A., and Lewis, M. (2021). Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*. 14
- Qin, Y., Wang, Y., Deng, D., Zhao, Z., Yang, X., Liu, L., Wei, S., Hu, Y., and Yin, S. (2023). Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14. 13
- Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. (2019). Compressive transformers for long-range sequence modelling. 14
- Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. (2022). Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International Conference on Machine Learning*, pages 18332–18346. PMLR. 9
- Rotem, D., Hassid, M., Mamou, J., and Schwartz, R. (2023). Finding the sweet spot: Analysis and improvement of adaptive inference in low resource settings. 8
- Sahu, G., Vechtomova, O., Bahdanau, D., and Laradji, I. H. (2023). Promptmix: A class boundary augmentation method for large language model distillation. *arXiv preprint arXiv:2310.14192*. 6

- Santilli, A., Severino, S., Postolache, E., Maiorca, V., Mancusi, M., Marin, R., and Rodolà, E. (2023). Accelerating transformer inference for translation via parallel decoding. *arXiv preprint arXiv:2305.10427*. 10
- Savinov, N., Chung, J., Binkowski, M., Elsen, E., and Oord, A. v. d. (2021). Step-unrolled denoising autoencoders for text generation. *arXiv preprint arXiv:2112.06749*. 10
- Scao, T. L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., Luccioni, A. S., Yvon, F., Gallé, M., et al. (2022). Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*. 1
- Schotthöfer, S., Zangrando, E., Kusch, J., Ceruti, G., and Tudisco, F. (2022). Low-rank lottery tickets: finding efficient low-rank neural networks via matrix differential equations. *Advances in Neural Information Processing Systems*, 35:20051–20063. 7
- Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V. Q., Tay, Y., and Metzler, D. (2022). Confident adaptive language modeling. 7, 8
- Schuster, T., Fisch, A., Jaakkola, T., and Barzilay, R. (2021). Consistent accelerated inference via confident adaptive transformers. 7
- Schwartz, R., Stanovsky, G., Swayamdipta, S., Dodge, J., and Smith, N. A. (2020). The right tool for the job: Matching model and instance complexities. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6640–6651, Online. Association for Computational Linguistics. 7
- Shang, Y., Duan, B., Zong, Z., Nie, L., and Yan, Y. (2021). Lipschitz continuity guided knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10675–10684. 6
- Shang, Y., Yuan, Z., and Dong, Z. (2024). Pb-llm: Partially binarized large language models. In *ICLR*. 6
- Shao, C., Zhang, J., Feng, Y., Meng, F., and Zhou, J. (2020). Minimizing the bag-of-ngrams difference for non-autoregressive neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 198–205. 10
- Sharma, P., Ash, J. T., and Misra, D. (2023). The truth is in there: Improving reasoning in language models with layer-selective rank reduction. *arXiv preprint arXiv:2312.13558*. 7
- Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. (2023). Flexgen: High-throughput generative inference of large language models with a single GPU. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J., editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116. PMLR. 11
- Simoulin, A. and Crabbé, B. (2021). How many layers and why? An analysis of the model depth in transformers. In Kabbara, J., Lin, H., Paullada, A., and Vamvas, J., editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: Student Research Workshop*, pages 221–228, Online. Association for Computational Linguistics. 7
- Song, Y., Meng, C., Liao, R., and Ermon, S. (2021). Accelerating feedforward computation via parallel nonlinear equation solving. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9791–9800. PMLR. 10
- Song, Y., Mi, Z., Xie, H., and Chen, H. (2023). Powerinfer: Fast large language model serving with a consumer-grade gpu. 8
- Stern, M., Shazeer, N., and Uszkoreit, J. (2018). Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31. 9
- Stickland, A. C. and Murray, I. (2019). Bert and pals: Projected attention layers for efficient adaptation in multi-task learning. 7
- Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. (2023a). A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*. 6
- Sun, X., Choi, J., Chen, C.-Y., Wang, N., Venkataramani, S., Srinivasan, V. V., Cui, X., Zhang, W., and Gopalakrishnan, K. (2019a). Hybrid 8-bit floating point (hfp8) training and inference for deep neural networks. *Advances in neural information processing systems*, 32. 12
- Sun, Y., Dong, L., Patra, B., Ma, S., Huang, S., Benhaim, A., Chaudhary, V., Song, X., and Wei, F. (2022). A length-extrapolatable transformer. 14
- Sun, Z., Li, Z., Wang, H., He, D., Lin, Z., and Deng, Z. (2019b). Fast structured decoding for sequence models. *Advances in Neural Information Processing Systems*, 32. 10
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. (2023b). Spectr: Fast speculative decoding via optimal transport. *arXiv preprint arXiv:2310.15141*. 9
- Sundar Pichai, D. H. (2024). Our next-generation model: Gemini 1.5. 6

- Tambe, T., Yang, E.-Y., Wan, Z., Deng, Y., Reddi, V. J., Rush, A., Brooks, D., and Wei, G.-Y. (2020). Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE. 13
- Team, G., Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., et al. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*. 13
- Tesla (2023). Tesla dojo technology: A guide to tesla’s configurable floating point formats & arithmetic. <https://cdn.motor1.com/pdf-files/535242876-tesla-dojotechnology.pdf/>. 13
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023a). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. 1, 4
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023b). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*. 1
- Vaidya, N., Oh, F., and Comly, N. (2023). Optimizing inference on large language models with nvidia tensorrt-llm, now publicly available. 11
- Valipour, M., Rezagholizadeh, M., Kobzyev, I., and Ghodsi, A. (2022). Dylora: Parameter efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. *arXiv preprint arXiv:2210.07558*. 6
- Wang, P., Wang, Z., Li, Z., Gao, Y., Yin, B., and Ren, X. (2023). Scott: Self-consistent chain-of-thought distillation. *arXiv preprint arXiv:2305.01879*. 6
- Wang, W., Chen, W., Luo, Y., Long, Y., Lin, Z., Zhang, L., Lin, B., Cai, D., and He, X. (2024a). Model compression and efficient inference for large language models: A survey. *arXiv preprint arXiv:2402.09748*. 2
- Wang, W., Dong, L., Cheng, H., Liu, X., Yan, X., Gao, J., and Wei, F. (2024b). Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems*, 36. 14
- Wang, Y., Tian, F., He, D., Qin, T., Zhai, C., and Liu, T.-Y. (2019). Non-autoregressive machine translation with auxiliary regularization. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 5377–5384. 10
- Wei, B., Wang, M., Zhou, H., Lin, J., Xie, J., and Sun, X. (2019). Imitation learning for non-autoregressive neural machine translation. *arXiv preprint arXiv:1906.02041*. 10
- Wei, H., Kong, L., Chen, J., Zhao, L., Ge, Z., Yu, E., Sun, J., Han, C., and Zhang, X. (2024). Small language model meets with reinforced vision vocabulary. *arXiv preprint arXiv:2401.12503*. 14
- Wu, M., Waheed, A., Zhang, C., Abdul-Mageed, M., and Aji, A. F. (2023a). Lamini-lm: A diverse herd of distilled models from large-scale instructions. *arXiv preprint arXiv:2304.14402*. 6
- Wu, Q., Lan, Z., Qian, K., Gu, J., Geramifard, A., and Yu, Z. (2020). Memformer: A memory-augmented transformer for sequence modeling. *arXiv preprint arXiv:2010.06891*. 14
- Wu, S., Chen, H., Quan, X., Wang, Q., and Wang, R. (2023b). Ad-kd: Attribution-driven knowledge distillation for language model compression. *arXiv preprint arXiv:2305.10010*. 6
- Wu, X., Xia, H., Youn, S., Zheng, Z., Chen, S., Bakhtiari, A., Wyatt, M., He, Y., Ruwase, O., Song, L., et al. (2023c). Zeroquant (4+ 2): Redefining llms quantization with a new fp6-centric strategy for diverse generative tasks. *arXiv preprint arXiv:2312.08583*. 13
- Wu, X., Yao, Z., and He, Y. (2023d). Zeroquant-fp: A leap forward in llms post-training w4a8 quantization using floating-point formats. *arXiv preprint arXiv:2307.09782*. 6, 13
- Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. (2022). Memorizing transformers. *arXiv preprint arXiv:2203.08913*. 14
- Xia, H., Zheng, Z., Li, Y., Zhuang, D., Zhou, Z., Qiu, X., Li, Y., Lin, W., and Song, S. L. (2023a). Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *arXiv preprint arXiv:2309.10285*. 6
- Xia, H., Zheng, Z., Wu, X., Chen, S., Yao, Z., Youn, S., Bakhtiari, A., Wyatt, M., Zhuang, D., Zhou, Z., et al. (2024). Fp6-llm: Efficiently serving large language models through fp6-centric algorithm-system co-design. *arXiv preprint arXiv:2401.14112*. 13
- Xia, M., Gao, T., Zeng, Z., and Chen, D. (2023b). Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694*. 6
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. (2023a). Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR. 6
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2023b). Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*. 14

- Xiao, Y., Wu, L., Guo, J., Li, J., Zhang, M., Qin, T., and Liu, T.-y. (2023c). A survey on non-autoregressive generation for neural machine translation and beyond. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 10
- Xin, J., Tang, R., Lee, J., Yu, Y., and Lin, J. (2020). Deebert: Dynamic early exiting for accelerating bert inference. 7
- Xiong, W., Liu, J., Molybog, I., Zhang, H., Bhargava, P., Hou, R., Martin, L., Rungta, R., Sankararaman, K. A., Oguz, B., Khabsa, M., Fang, H., Mehdad, Y., Narang, S., Malik, K., Fan, A., Bhosale, S., Edunov, S., Lewis, M., Wang, S., and Ma, H. (2023). Effective long-context scaling of foundation models. 14
- Xu, D., Yin, W., Jin, X., Zhang, Y., Wei, S., Xu, M., and Liu, X. (2023a). Llmcad: Fast and scalable on-device large language model inference. *arXiv preprint arXiv:2309.04255*. 9
- Xu, M., Xu, Y. L., and Mandic, D. P. (2023b). Tensorgpt: Efficient compression of the embedding layer in llms based on the tensor-train decomposition. *arXiv preprint arXiv:2307.00526*. 7
- Xu, X., Li, M., Tao, C., Shen, T., Cheng, R., Li, J., Xu, C., Tao, D., and Zhou, T. (2024). A survey on knowledge distillation of large language models. 6
- Xu, Y., Xie, L., Gu, X., Chen, X., Chang, H., Zhang, H., Chen, Z., Zhang, X., and Tian, Q. (2023c). Qalora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*. 6
- Yang, N., Ge, T., Wang, L., Jiao, B., Jiang, D., Yang, L., Majumder, R., and Wei, F. (2023). Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*. 10
- Yao, Z., Yazdani Aminabadi, R., Zhang, M., Wu, X., Li, C., and He, Y. (2022). Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35:27168–27183. 6
- Yi, R., Guo, L., Wei, S., Zhou, A., Wang, S., and Xu, M. (2023). Edgemoe: Fast on-device inference of moe-based large language models. *arXiv preprint arXiv:2308.14352*. 9
- Yin, L., Wu, Y., Zhang, Z., Hsieh, C.-Y., Wang, Y., Jia, Y., Pechenizkiy, M., Liang, Y., Wang, Z., and Liu, S. (2023a). Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity. *arXiv preprint arXiv:2310.05175*. 6
- Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., and Chen, E. (2023b). A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549*. 13
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. (2022). Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. 11
- Yuan, Z., Li, Z., and Sun, L. (2023a). Tinygpt-v: Efficient multimodal large language model via small backbones. *arXiv preprint arXiv:2312.16862*. 14
- Yuan, Z., Liu, J., Wu, J., Yang, D., Wu, Q., Sun, G., Liu, W., Wang, X., and Wu, B. (2023b). Benchmarking the reliability of post-training quantization: a particular focus on worst-case performance. *arXiv preprint arXiv:2303.13003*. 13
- Yuan, Z., Niu, L., Liu, J., Liu, W., Wang, X., Shang, Y., Sun, G., Wu, Q., Wu, J., and Wu, B. (2023c). Rptq: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089*. 6, 13
- Yuan, Z., Shang, Y., Song, Y., Wu, Q., Yan, Y., and Sun, G. (2023d). Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821*. 7
- Yue, Y., Yuan, Z., Duanmu, H., Zhou, S., Wu, J., and Nie, L. (2024). Wkvquant: Quantizing weight and key/value cache for large language models gains more. *arXiv preprint arXiv:2402.12065*. 6
- Yuksel, S. E., Wilson, J. N., and Gader, P. D. (2012). Twenty years of mixture of experts. *IEEE Transactions on Neural Networks and Learning Systems*, 23(8):1177–1193. 8
- Zadeh, A. H. et al. (2020). Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *MICRO*, pages 811–824. IEEE. 13
- Zeng, S., Liu, J., Dai, G., Yang, X., Fu, T., Wang, H., Ma, W., Sun, H., Li, S., Huang, Z., et al. (2024). Flightllm: Efficient large language model inference with a complete mapping flow on fpga. *arXiv preprint arXiv:2401.03868*. 13
- Zeng, Z., Hong, Y., Dai, H., Zhuang, H., and Chen, C. (2023). Consistentee: A consistent and hardness-guided early exiting method for accelerating language models inference. 8
- Zhang, D., Yu, Y., Li, C., Dong, J., Su, D., Chu, C., and Yu, D. (2024a). Mm-llms: Recent advances in multimodal large language models. *arXiv preprint arXiv:2401.13601*. 13
- Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., and Mehrotra, S. (2023). Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*. 9

- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. (2022). Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*. 1
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. (2024b). H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36. 14
- Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., and Cui, B. (2024). Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473*. 14
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*. 1, 2
- Zhong, W., Guo, L., Gao, Q., and Wang, Y. (2023). Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*. 14
- Zhou, B., Hu, Y., Weng, X., Jia, J., Luo, J., Liu, X., Wu, J., and Huang, L. (2024). Tinyllava: A framework of small-scale large multimodal models. *arXiv preprint arXiv:2402.14289*. 14
- Zhou, J., Wu, J., Gao, Y., Ding, Y., Tao, C., Li, B., Tu, F., Cheng, K.-T., So, H. K.-H., and Wong, N. (2023a). Dybit: Dynamic bit-precision numbers for efficient quantized neural network inference. *arXiv preprint arXiv:2302.12510*. 13
- Zhou, W., Jiang, Y. E., Cui, P., Wang, T., Xiao, Z., Hou, Y., Cotterell, R., and Sachan, M. (2023b). Recurrentgpt: Interactive generation of (arbitrarily) long text. *arXiv preprint arXiv:2305.13304*. 14
- Zhou, W., Xu, C., Ge, T., McAuley, J., Xu, K., and Wei, F. (2020). Bert loses patience: Fast and robust inference with early exit. 7
- Zhou, Y., Lyu, K., Rawat, A. S., Menon, A. K., Rostamizadeh, A., Kumar, S., Kagy, J.-F., and Agarwal, R. (2023c). Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*. 9
- Zhou, Z., Li, C., Yang, F., and Suny, G. (2023d). Dimm-link: Enabling efficient inter-dimm communication for near-memory processing. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 302–316. IEEE. 12
- Zhu, W. (2021). LeeBERT: Learned early exit for BERT with cross-level optimization. In Zong, C., Xia, F., Li, W., and Navigli, R., editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2968–2980, Online. Association for Computational Linguistics. 7
- Zhu, X., Li, J., Liu, Y., Ma, C., and Wang, W. (2023). A survey on model compression for large language models. *arXiv preprint arXiv:2308.07633*. 1
- Zhu, Y., Zhu, M., Liu, N., Ou, Z., Mou, X., and Tang, J. (2024). Llava-phi: Efficient multi-modal assistant with small language model. *arXiv preprint arXiv:2401.02330*. 14
- Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. (2022). St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*. 9